

1.	Prerequisites:.....	3
2.	Installation of Kafka – 90 Mins	5
3.	Installation Confluent Kafka (Local) – 30 Minutes.....	13
4.	Basic Kafka Operations - CLI (Topic) – 30 Mins	15
5.	Basic Kafka Operations - CLI (Producer & Consumer) – 30 Mins	21
6.	Zookeeper – 120 Minutes.....	28
7.	Kafka 3 Nodes - cluster – 90 Minutes.....	47
8.	Securing Kafka SSL – 90 Minutes	68
9.	Securing Kafka ACL – 60 Minutes.....	90
10.	Schema Registry - Manage Schemas for Topics – 30 Minutes	111
11.	Kafka Connector (JDBC) - 120 Minutes	119
12.	Stream Kafka topic data into Elasticsearch – 60 Minutes	139

Hardware:

8 GB RAM , 30 GB HDD , Centos 7 or above OS. Access to internet.

Last Updated: September 16 – 2026

Software:

- https://archive.apache.org/dist/kafka/3.9.0/kafka_2.13-3.9.0.tgz
- jdk 11.0.1
- apache-zookeeper-3.6.3-bin.tar.gz
- confluent-8.3.0.tar
- Eclipse for Linux. (Any Latest version for JEE Development)

1. Prerequisites:

Using docker:

Steps to be performed:

- Instantiate a container, kafka0.
- You can copy files from the host to container using docker copy command.

Execute the following command, it will perform the following:

- Create a network : spark-net
- download the image, centos:8
- start a container with the name, kafka0 and mount host folder in /opt

```
#docker network create --driver bridge spark-net
```

ARM Architecture – mac with Mac Chipset

```
#docker run --name kafka0 --hostname kafka0 -p 9092:9092 -p 8081:8081 -p  
2181:2181 -p 9999:9999 -i -t --privileged --network spark-net -v  
/Users/henrypotsangbam/Documents/Software:/Software arm64v8/oraclelinux:9  
/usr/sbin/init
```

For Intel Chipset

```
#docker run --name kafka0 --hostname kafka0 -p 9092:9092 -p 8081:8081 -p
2181:2181 -p 9999:9999 -i -t --privileged --network spark-net -v
/Users/henrypotsangbam/Documents/Software:/Software oraclelinux:9 /usr/sbin/init
```

Note: **/Users/henrypotsangbam/Documents/Software** replace this directory with that of your system. Any folder to store software. E.x /tmp

You can verify the container from a separate terminal:

```
(base) Henrys-MacBook-Air:~ henrypotsangbam$ docker ps
```

CONTAINER ID	IMAGE	COMMAND NAMES	CREATED	STATUS
0945477d457b	ehenry0227/learning:spark-kafka-raw	"/bin/bash"	34 seconds ago	Up 33 secon
ds	0.0.0.0:6062->6062/tcp, 0.0.0.0:8081->8081/tcp	kafka0		

```
(base) Henrys-MacBook-Air:~ henrypotsangbam$
```

Perform the installation after this as a normal server.

Ends Here -----

2. Installation of Kafka – 90 Mins

Start kafka node and perform the following.
You need to install java before installing zookeeper and Kafka.

Installation of Java.

Extract jdk in /opt folder. Required software should be in /Software folder. Ensure to enter the appropriate jdk file of your system. Rename the jdk folder to jdk

```
#tar -xvf jdk-11.0.16_linux* -C /opt  
#cd /opt  
#mv jdk* jdk
```

Set the above path in the PATH variable and JAVA_HOME

Update profile as follow:

```
#vi ~/.bashrc  
  
export JAVA_HOME=/opt/jdk  
export PATH=$PATH:$JAVA_HOME/bin
```

Save the file and exit.

Update the shell scripts using the following command.

```
#bash
```

Install Zookeeper

You can choose any of the options given below:

Option I (Fresh Installation): (We will use this for our lab)

The following steps install Zookeeper with a basic configuration in /opt/zookeeper. Its configured to store data in /opt/data/zookeeper:

Extract the zookeeper archive file in /opt and rename the installation folder for brevity.

```
# tar -xvf apache-zookeeper* -C /opt
# cd /opt
#mv a*zookeeper* /opt/zookeeper
#mkdir -p /opt/data/zookeeper
```

Create a zookeeper configuration file and update with the following values.

```
#vi /opt/zookeeper/conf/zoo.cfg
tickTime=2000
dataDir=/opt/data/zookeeper
clientPort=2181
```

Update the zoo.cfg with the above entries. You can save the file using esc+wq!

Start the zookeeper using the following scripts.

```
# /opt/zookeeper/bin/zkServer.sh start
```

```
[root@tos opt]# /opt/zookeeper/bin/zkServer.sh start
ZooKeeper JMX enabled by default
Using config: /opt/zookeeper/bin/../conf/zoo.cfg
Starting zookeeper ... STARTED
[root@tos opt]#
```

Option II (Part of the Apache Kafka) – Skip.

```
#bin/zookeeper-server-start.sh config/zookeeper.properties
```

Option II Ends Here.

You can now validate that Zookeeper is running correctly in standalone mode by connecting to the client port and sending the four-letter command srvr:

```
#yum install telnet
```

```
#telnet localhost 2181
```

Enter the following in the zookeeper CLI.

```
srvr
```

```
[root@tos opt]# telnet localhost 2181
Trying ::1...
Connected to localhost.
Escape character is '^]'.
srvr
Zookeeper version: 3.4.12-e5259e437540f349646870ea94dc2658c4e44b3b, built on 03/
27/2018 03:55 GMT
Latency min/avg/max: 0/0/0
Received: 2
Sent: 1
Connections: 1
Outstanding: 0
Zxid: 0x0
Mode: standalone
Node count: 4
Connection closed by foreign host.
[root@tos opt]#
```

After zookeeper installation, let us install the Kafka Broker.

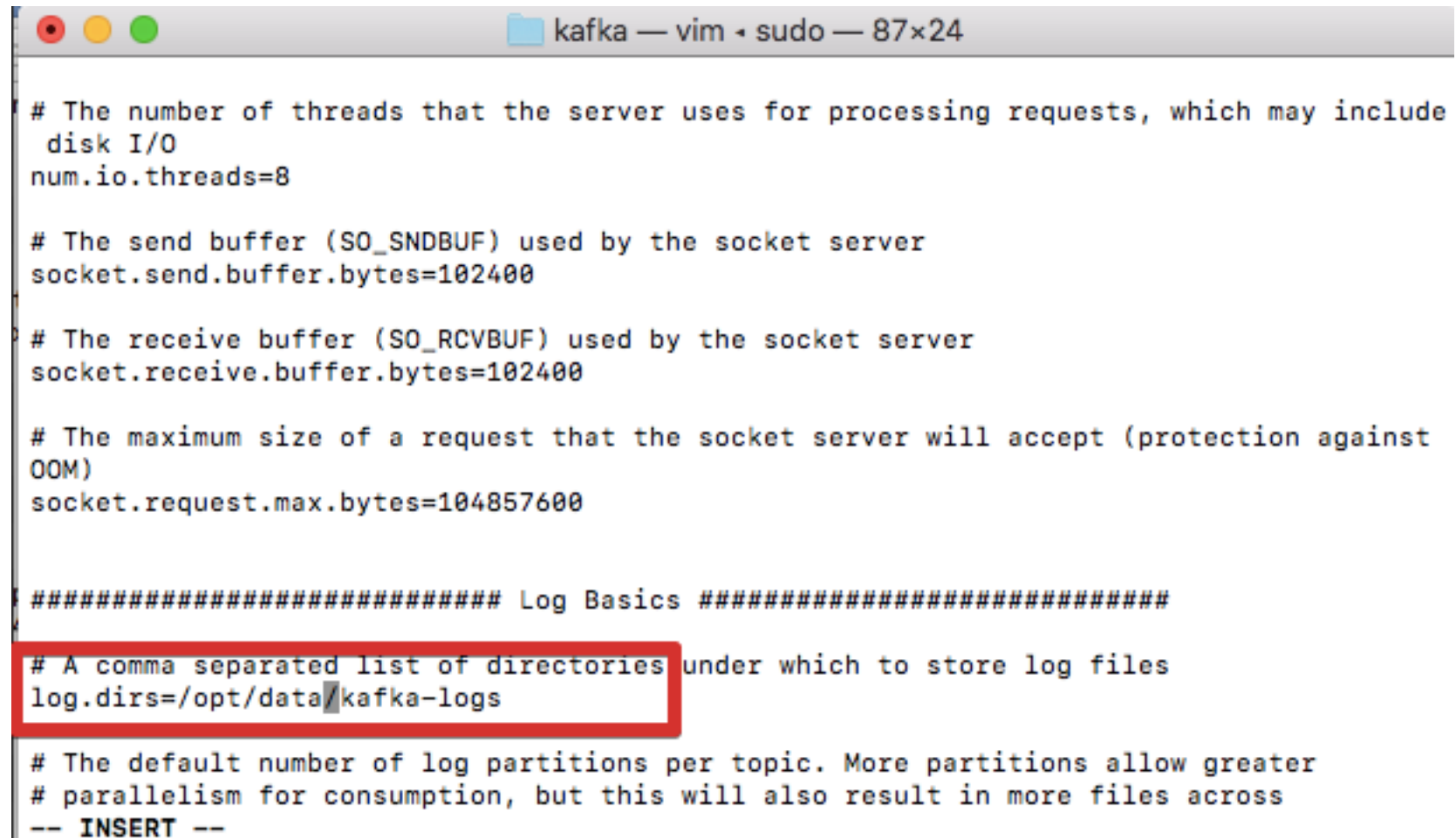
Installation of Kafka Broker

The following example installs Kafka in /opt/kafka, configured to use the Zookeeper server started previously and to store the message log segments stored in /tmp/kafka-logs: You need to execute the following command from the folder where the software is located.

```
# tar -xvf kafka* -C /opt
#cd /opt
```

```
# mv kafka* /opt/kafka
# mkdir /opt/data/kafka-logs
```

Update `/opt/kafka/config/server.properties` to store Kafka Log in the above mention folder.



```
kafka — vim — sudo — 87x24

# The number of threads that the server uses for processing requests, which may include
# disk I/O
num.io.threads=8

# The send buffer (SO_SNDBUF) used by the socket server
socket.send.buffer.bytes=102400

# The receive buffer (SO_RCVBUF) used by the socket server
socket.receive.buffer.bytes=102400

# The maximum size of a request that the socket server will accept (protection against
# OOM)
socket.request.max.bytes=104857600

##### Log Basics #####

# A comma separated list of directories under which to store log files
log.dirs=/opt/data/kafka-logs

# The default number of log partitions per topic. More partitions allow greater
# parallelism for consumption, but this will also result in more files across
-- INSERT --
```

Back Up the configuration for later.

```
#cp /opt/kafka/config/server.properties /opt/kafka/config/server.properties_plain
```

Start the broker with the following command

```
# /opt/kafka/bin/kafka-server-start.sh -daemon /opt/kafka/config/server.properties
```

```
[root@tos opt]# /opt/kafka/bin/kafka-server-start.sh -daemon /opt/kafka/config/s
erver.properties
[root@tos opt]# jps
3476 Kafka
3499 Jps
2895 QuorumPeerMain
[root@tos opt]#
```

```
#mkdir /opt/scripts
```

All the common execution scripts will be stored in the above folder.

The following scripts will start a zookeeper along with a broker. Create the following files and update with the following scripts. It will start the zookeeper and kafka broker using the mention script.

```
#cd /opt/scripts
```

```
#vi startABroker.sh
```

Enter the following in the above file.

```
##### Scripts Begin #####
#!/usr/bin/env bash
# Start Zookeeper.
/opt/zookeeper/bin/zkServer.sh start
#Start Kafka Server
/opt/kafka/bin/kafka-server-start.sh -daemon /opt/kafka/config/server.properties
echo "Started Successfully"
//##### Scripts End #####
```

To shutdown the Broker, find the process and kill.

```
ps -eaf | grep java
```

or

create a script:

```
#vi /opt/scripts/stopBroker.sh
```

with the following commands in it.

```
#!/usr/bin/env bash
```

```
/opt/kafka/bin/kafka-server-stop.sh
```

To Stop Zookeeper create the following script. Don't include the ---- line.

```
#vi /opt/scripts/stopZookeeper.sh
```

Update the following commands in the above script and save it.

```
-----
```

```
#!/usr/bin/env bash
# Stop Zookeeper.
/opt/zookeeper/bin/zkServer.sh stop
echo "Stop zookeeper Successfully"
```

To reinitialize the Cluster.

```
#vi reinitializeCluster.sh
```

```
#!/usr/bin/env bash
rm -fr /opt/kafka/data/kafka-logs/*
rm -fr /opt/kafka/data/zookeeper/*
cp /opt/kafka/config/server.properties_plain /opt/kafka/config/server.properties
echo "Reinitialize Successfully"
```

Lab Installation completes End here.

3. Installation Confluent Kafka (Local) – 30 Minutes

The purpose of this lab is to demonstrate the basic and most powerful capabilities of Confluent Platform – Schema Registry.

Install Confluent kafka with the following step.

Inflate the confluent kafka compress file as shown below. Software should be in /Software

You can use 8.3.3 version too.

```
#tar -xvf confluent-7.2.2.tar -C /opt
```

Rename the folder.

```
#cd /opt
```

```
#mv confluent-7.2.2 confluent
```

Set the environment variable for the Confluent Platform directory.

```
export CONFLUENT_HOME=/opt/confluent
```

Set your PATH variable:

```
# vi ~/.bashrc
```

```
export PATH=/opt/confluent/bin:${PATH};
```

type the following to activate the script.

#bash

----- Lab Ends Here -----

4. Basic Kafka Operations - CLI (Topic) – 30 Mins

In this lab you will be able to create a topic and perform some operations to understand the information about topic like partition and replication.

You need to start the broker using startABroker.sh. The script should be in /opt/scripts folder

```
#sh startABroker.sh
```

```
#jps
```

```
[root@tos scripts]# sh startABroker.sh
ZooKeeper JMX enabled by default
Using config: /opt/zookeeper/bin/./conf/zoo.cfg
Starting zookeeper ... STARTED
Started Successfully
[root@tos scripts]# jps
11665 Jps
11646 Kafka
11375 QuorumPeerMain
[root@tos scripts]#
```

Once the Kafka broker is started, we can verify that it is working by performing some simple operations against the broker; creating a test topic etc.

Create and verify details about topic:

```
# /opt/kafka/bin/kafka-topics.sh --create --bootstrap-server kafka0:9092 --
replication-factor 1 --partitions 1 --topic test
```

```
[root@kafka0 scripts]# /opt/kafka/bin/kafka-topics.sh --create --bootstrap-server kafka0:9092 --replication-factor 1 --partitions 1 --topic test
Created topic test.
```

```
# /opt/kafka/bin/kafka-topics.sh --create --bootstrap-server kafka0:9092 --
replication-factor 1 --partitions 12 --topic IBM
```

```
# /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka0:9092 --describe --topic
test
```

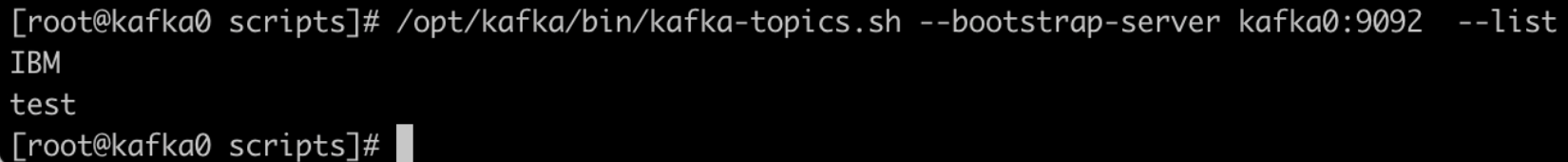
```
#/opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka0:9092 --describe --topic
IBM
```

```
[root@kafka0 scripts]# /opt/kafka/bin/kafka-topics.sh --create --bootstrap-server kafka0:9092 --replication-factor 1 --partitions 12 --topic IBM
Created topic IBM.
[root@kafka0 scripts]# /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka0:9092 --describe --topic test
Topic: test      TopicId: 9Rreu1aUR6uHfz3GHnYhtA PartitionCount: 1      ReplicationFactor: 1      Configs: segment.bytes=1073741824
Topic: test      Partition: 0      Leader: 0      Replicas: 0      Isr: 0
[root@kafka0 scripts]# /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka0:9092 --describe --topic IBM
Topic: IBM       TopicId: eY3Tl3N_T6iYY_ZzV70Pjg PartitionCount: 12      ReplicationFactor: 1      Configs: segment.bytes=1073741824
Topic: IBM       Partition: 0      Leader: 0      Replicas: 0      Isr: 0
Topic: IBM       Partition: 1      Leader: 0      Replicas: 0      Isr: 0
Topic: IBM       Partition: 2      Leader: 0      Replicas: 0      Isr: 0
Topic: IBM       Partition: 3      Leader: 0      Replicas: 0      Isr: 0
Topic: IBM       Partition: 4      Leader: 0      Replicas: 0      Isr: 0
Topic: IBM       Partition: 5      Leader: 0      Replicas: 0      Isr: 0
Topic: IBM       Partition: 6      Leader: 0      Replicas: 0      Isr: 0
Topic: IBM       Partition: 7      Leader: 0      Replicas: 0      Isr: 0
Topic: IBM       Partition: 8      Leader: 0      Replicas: 0      Isr: 0
Topic: IBM       Partition: 9      Leader: 0      Replicas: 0      Isr: 0
Topic: IBM       Partition: 10     Leader: 0      Replicas: 0      Isr: 0
Topic: IBM       Partition: 11     Leader: 0      Replicas: 0      Isr: 0
[root@kafka0 scripts]#
```

Verify the no of partition in the output above.

list and describe topic.

```
#/opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka0:9092 --list
```

A terminal window screenshot showing the command `/opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka0:9092 --list` being executed. The output shows two topics: `IBM` and `test`. The prompt is `[root@kafka0 scripts]#`.

```
[root@kafka0 scripts]# /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka0:9092 --list
IBM
test
[root@kafka0 scripts]#
```

List and describe Topics

What does the tool do?

This tool lists the information for a given list of topics. If no topics are provided in the command line, the tool queries zookeeper to get all the topics and lists the information for them. The fields that the tool displays are - topic name, partition, leader, replicas, isr.

How to use the tool?

List only single topic named "test" (prints only topic name)

```
#/opt/kafka/bin/kafka-topics.sh --list --bootstrap-server kafka0:9092 --topic test
```

List all topics (prints only topic names)

```
#!/opt/kafka/bin/kafka-topics.sh --list --bootstrap-server kafka0:9092
```

```
[root@kafka0 scripts]# /opt/kafka/bin/kafka-topics.sh --list --bootstrap-server kafka0:9092 --topic test
test
[root@kafka0 scripts]# /opt/kafka/bin/kafka-topics.sh --list --bootstrap-server kafka0:9092
IBM
test
[root@kafka0 scripts]#
```

Describe only single topic named "test" (prints details about the topic)

```
#!/opt/kafka/bin/kafka-topics.sh --describe --bootstrap-server kafka0:9092 --topic
test
```

Describe all topics (prints details about the topics)

```
#!/opt/kafka/bin/kafka-topics.sh --describe --bootstrap-server kafka0:9092
```

```
[root@kafka0 scripts]# /opt/kafka/bin/kafka-topics.sh --describe --bootstrap-server kafka0:9092 --topic test
Topic: test      TopicId: 9Rreu1aUR6uHfz3GHnYhtA PartitionCount: 1      ReplicationFactor: 1      Configs: segment.bytes=1073741824
  Topic: test      Partition: 0      Leader: 0      Replicas: 0      Isr: 0
[root@kafka0 scripts]# /opt/kafka/bin/kafka-topics.sh --describe --bootstrap-server kafka0:9092
Topic: test      TopicId: 9Rreu1aUR6uHfz3GHnYhtA PartitionCount: 1      ReplicationFactor: 1      Configs: segment.bytes=1073741824
  Topic: test      Partition: 0      Leader: 0      Replicas: 0      Isr: 0
Topic: IBM       TopicId: eY3Tl3N_T6iYY_ZzV70Pjg PartitionCount: 12      ReplicationFactor: 1      Configs: segment.bytes=1073741824
  Topic: IBM       Partition: 0      Leader: 0      Replicas: 0      Isr: 0
  Topic: IBM       Partition: 1      Leader: 0      Replicas: 0      Isr: 0
  Topic: IBM       Partition: 2      Leader: 0      Replicas: 0      Isr: 0
  Topic: IBM       Partition: 3      Leader: 0      Replicas: 0      Isr: 0
  Topic: IBM       Partition: 4      Leader: 0      Replicas: 0      Isr: 0
  Topic: IBM       Partition: 5      Leader: 0      Replicas: 0      Isr: 0
  Topic: IBM       Partition: 6      Leader: 0      Replicas: 0      Isr: 0
  Topic: IBM       Partition: 7      Leader: 0      Replicas: 0      Isr: 0
  Topic: IBM       Partition: 8      Leader: 0      Replicas: 0      Isr: 0
  Topic: IBM       Partition: 9      Leader: 0      Replicas: 0      Isr: 0
  Topic: IBM       Partition: 10     Leader: 0      Replicas: 0      Isr: 0
  Topic: IBM       Partition: 11     Leader: 0      Replicas: 0      Isr: 0
[root@kafka0 scripts]#
```

We will understand the output in details later.

Create Topics

What does the tool do?

By default, Kafka auto creates topic if "auto.create.topics.enable" is set to true on the server. This creates a topic with a default number of partitions, replication factor and uses Kafka's default scheme to do replica assignment. Sometimes, it may be required that we would like to customize a topic while creating it. This tool helps to create a topic and also specify the number of partitions, replication factor and replica assignment list for the topic.

How to use the tool?

create topic with default settings

```
#/opt/kafka/bin/kafka-topics.sh --create --bootstrap-server kafka0:9092 --topic
topic1 --partitions 2 --replication-factor 1
```

```
[root@tos scripts]# /opt/kafka/bin/kafka-topics.sh --create --zookeeper localhost:2181 --topic to
pic1 --partitions 2 --replication-factor 1
Created topic "topic1".
[root@tos scripts]#
```

Create a topic with replication 2.

```
#/opt/kafka/bin/kafka-topics.sh --create --bootstrap-server kafka0:9092 --topic
topic2 --partitions 2 --replication-factor 2
```

```
[root@kafka0 scripts]# /opt/kafka/bin/kafka-topics.sh --create --bootstrap-server kafka0:9092 --topic topic1 --partitions 2 --replication-factor 1
Created topic topic1.
[root@kafka0 scripts]# /opt/kafka/bin/kafka-topics.sh --create --bootstrap-server kafka0:9092 --topic topic2 --partitions 2 --replication-factor 2
Error while executing topic command : Replication factor: 2 larger than available brokers: 1.
[2023-07-03 15:53:18,020] ERROR org.apache.kafka.common.errors.InvalidReplicationFactorException: Replication factor: 2 larger than available brokers: 1.
(kafka.admin.TopicCommand$)
[root@kafka0 scripts]#
```

As shown above, it generates an error. Since there is only a single broker. It will fix later.

-----Lab CLI completes End here-----

5. Basic Kafka Operations - CLI (Producer & Consumer) – 30 Mins

In this lab we will send message to the broker and consumer message using the kafka inbuilt commands.

You need to complete the previous lab before proceeding ahead.

You need to start the broker using startABroker.sh if not done earlier. The script should be in /opt/scripts folder

```
#sh startABroker.sh
```

```
#jps
```

```
[root@tos scripts]# sh startABroker.sh
ZooKeeper JMX enabled by default
Using config: /opt/zookeeper/bin/../conf/zoo.cfg
Starting zookeeper ... STARTED
Started Successfully
[root@tos scripts]# jps
11665 Jps
11646 Kafka
11375 QuorumPeerMain
[root@tos scripts]#
```

Sent message to **test** topic: Open a console to send message to the topic, test. Enter some text as shown below.

```
# /opt/kafka/bin/kafka-console-producer.sh --broker-list kafka0:9092 --topic test
```

Test Message 1

Test Message 2

^D

```
#
```

```
[root@kafka0 scripts]#
```

```
[root@kafka0 scripts]# /opt/kafka/bin/kafka-console-producer.sh --broker-list kafka0:9092 --topic test
```

```
>Test Message 1
```

```
>Test Message 2
```

```
>
```

Consume messages from a test topic: As soon as you enter the following script in a separate terminal, you should be able to consume the messages that we have type in the producer console.

```
# /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server kafka0:9092 --topic test --from-beginning
```

```
[root@kafka0 scripts]# /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server kafka0:9092 --topic test --from-beginning
```

```
Test Message 1
```

```
Test Message 2
```

DumplogSegment

You can also view the message publish to a topic from a log file.

```
/opt/kafka/bin/kafka-run-class.sh kafka.tools.DumpLogSegments --deep-iteration --print-data-log --files \
/opt/data/kafka-logs/test-0/00000000000000000000.log | head -n 12
```

```
[root@kafka0 scripts]# /opt/kafka/bin/kafka-run-class.sh kafka.tools.DumpLogSegments --deep-iteration --print-data-log --files /opt/data/kafka-logs/test-0/00000000000000000000.log | head -n 12
Dumping /opt/data/kafka-logs/test-0/00000000000000000000.log
Starting offset: 0
baseOffset: 0 lastOffset: 0 count: 1 baseSequence: 0 lastSequence: 0 producerId: 0 producerEpoch: 0 partitionLeaderEpoch: 0 isTransactional: false isControl: false deleteHorizonMs: OptionalLong.empty position: 0 CreateTime: 1688379972311 size: 82 magic: 2 compresscodec: none crc: 1391971847 invalid: true
| offset: 0 CreateTime: 1688379972311 keySize: -1 valueSize: 14 sequence: 0 headerKeys: [] payload: Test Message 1
baseOffset: 1 lastOffset: 1 count: 1 baseSequence: 1 lastSequence: 1 producerId: 0 producerEpoch: 0 partitionLeaderEpoch: 0 isTransactional: false isControl: false deleteHorizonMs: OptionalLong.empty position: 82 CreateTime: 1688379975484 size: 82 magic: 2 compresscodec: none crc: 2251353974 invalid: true
| offset: 1 CreateTime: 1688379975484 keySize: -1 valueSize: 14 sequence: 1 headerKeys: [] payload: Test Message 2
[root@kafka0 scripts]#
```

Determine the offset of the messages.
CLI to list offset number.

New kafka

```
/opt/kafka/bin/kafka-get-offsets.sh \ --bootstrap-server kafka0:9092 \ --topic test
```

Old Kafka.

```
#!/opt/kafka/bin/kafka-run-class.sh kafka.tools.GetOffsetShell --broker-list  
kafka0:9092 --topic test
```

```
[root@kafka0 scripts]# /opt/kafka/bin/kafka-run-class.sh kafka.tools.GetOffsetShell --broker-list kafka0:9092 --topic test  
test:0:2  
[root@kafka0 scripts]#
```

Partition 0 offset 2.

Sending Message with Key and Value

```
#!/opt/kafka/bin/kafka-console-producer.sh --broker-list kafka0:9092 --topic IBM --  
property "parse.key=true" --property "key.separator=:"
```

Enter

WAS:600

DB2:700

ESB:900

```
[root@kafka0 scripts]# /opt/kafka/bin/kafka-console-producer.sh --broker-list kafka0:9092 --topic IBM --property "parse.key=true" --property "key.separator=:"  
>  
>WAS:600  
>DB2:700  
>ESB:900  
>
```

```
#!/opt/kafka/bin/kafka-console-consumer.sh --topic IBM --bootstrap-server  
kafka0:9092 --from-beginning \
```

```
--property print.key=true \  
--property key.separator=":"
```

```
[root@kafka0 /]# /opt/kafka/bin/kafka-console-consumer.sh --topic IBM --bootstrap-server kafka0:9092 --from-beginning \  
> --property print.key=true \  
> --property key.separator=":"  
WAS:600  
ESB:900  
DB2:700
```

Let us understand consumer group load balancing concepts.

- Create a topic with 2 partitions – **city_info**.
- Initiate 2 consumers with same consumer Group
- Send 6 messages to the topic
- Verify load get distributed among the 2 consumers.

```
#/opt/kafka/bin/kafka-topics.sh --create --bootstrap-server kafka0:9092 --topic  
city_info --partitions 2 --replication-factor 1
```

Initiate 2 consumers with same consumer Group
Open in a separate 2 terminals.

```
#/opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server kafka0:9092 --topic  
city_info --consumer-property group.id=citygroup
```

```
#/opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server kafka0:9092 --topic city_info --consumer-property group.id=citygroup
```

- Produce about 6 messages to the topic

```
#/opt/kafka/bin/kafka-console-producer.sh --broker-list kafka0:9092 --topic city_info --property "parse.key=true" --property "key.separator=:"
```

```
>NM:Nav1 Mumbai
>^C[root@kafka0 opt]# /opt/kafka/bin/kafka-console-producer.sh --broker-list kafka0:9092 --topic city_info --property "parse.key=true" --property "key.separator=:"
>MH:Mumbai
>MH:navi Mumbai
>DH:Delhi
>MN:Imphal
>KA:bangalore
>KR:Kochi
>KA:Mysore
>AM:Guwahati
>GT:Gujarat
>
```

- load should get distributed among the 2 consumers.

-----Lab CLI ends here-----

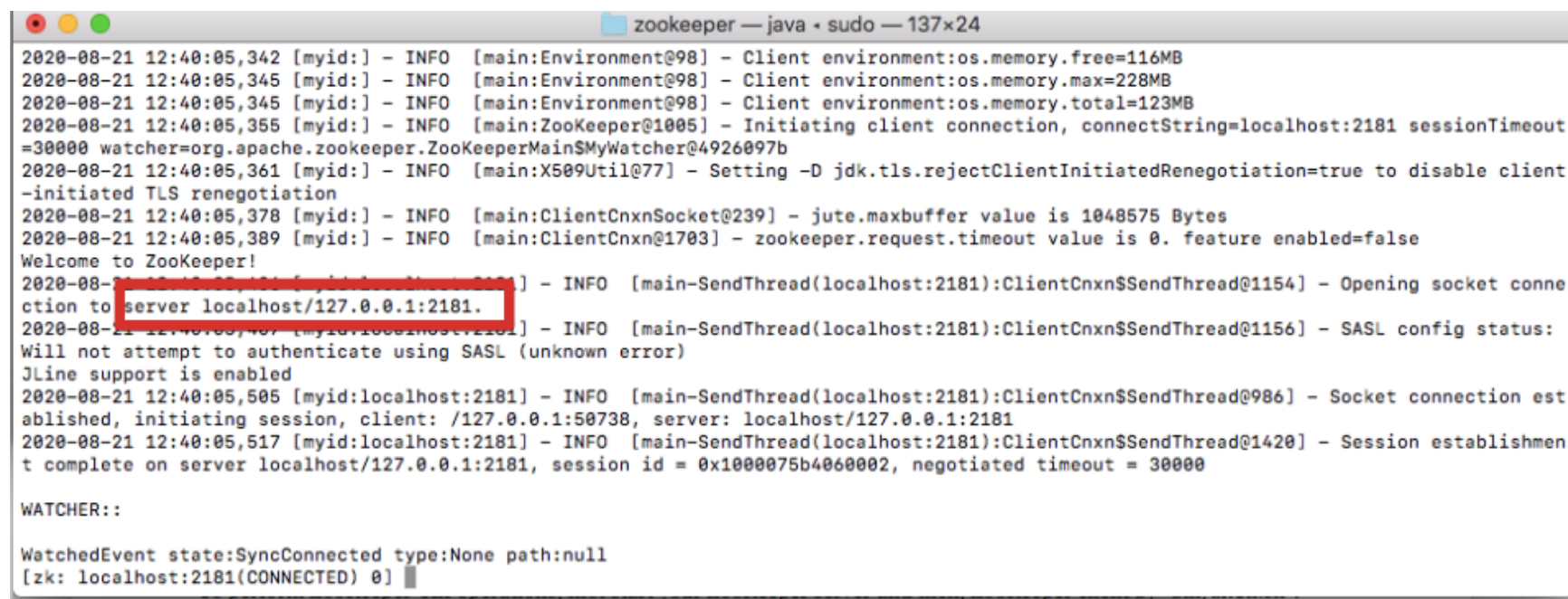
6. Zookeeper – 120 Minutes

In this lab, we will perform familiarization of ZK cli features and configure zookeeper ensemble:

To perform ZooKeeper CLI operations, first start your ZooKeeper server and then, ZooKeeper client by executing “bin/zkCli.sh”, which is in the Broker installation folder.

```
#cd /opt/zookeeper
```

```
#bin/zkCli.sh
```



```
zookeeper — java • sudo — 137x24
2020-08-21 12:40:05,342 [myid:] - INFO [main:Environment@98] - Client environment:os.memory.free=116MB
2020-08-21 12:40:05,345 [myid:] - INFO [main:Environment@98] - Client environment:os.memory.max=228MB
2020-08-21 12:40:05,345 [myid:] - INFO [main:Environment@98] - Client environment:os.memory.total=123MB
2020-08-21 12:40:05,355 [myid:] - INFO [main:ZooKeeper@1005] - Initiating client connection, connectString=localhost:2181 sessionTimeout=30000 watcher=org.apache.zookeeper.ZooKeeperMain$MyWatcher@4926097b
2020-08-21 12:40:05,361 [myid:] - INFO [main:X509Util@77] - Setting -D jdk.tls.rejectClientInitiatedRenegotiation=true to disable client-initiated TLS renegotiation
2020-08-21 12:40:05,378 [myid:] - INFO [main:ClientCnxnSocket@239] - jute.maxbuffer value is 1048575 Bytes
2020-08-21 12:40:05,389 [myid:] - INFO [main:ClientCnxn@1703] - zookeeper.request.timeout value is 0. feature enabled=false
Welcome to ZooKeeper!
2020-08-21 12:40:05,401 [myid:] - INFO [main-SendThread(localhost:2181):ClientCnxn$SendThread@1154] - Opening socket connection to server localhost/127.0.0.1:2181.
2020-08-21 12:40:05,407 [myid:] - INFO [main-SendThread(localhost:2181):ClientCnxn$SendThread@1156] - SASL config status: Will not attempt to authenticate using SASL (unknown error)
JLine support is enabled
2020-08-21 12:40:05,505 [myid:localhost:2181] - INFO [main-SendThread(localhost:2181):ClientCnxn$SendThread@986] - Socket connection established, initiating session, client: /127.0.0.1:50738, server: localhost/127.0.0.1:2181
2020-08-21 12:40:05,517 [myid:localhost:2181] - INFO [main-SendThread(localhost:2181):ClientCnxn$SendThread@1420] - Session establishment complete on server localhost/127.0.0.1:2181, session id = 0x1000075b4060002, negotiated timeout = 30000

WATCHER::

WatchedEvent state:SyncConnected type:None path:null
[zk: localhost:2181(CONNECTED) 0]
```

or New CLI

```
[root@kafka0 kafka]# bin/zookeeper-shell.sh localhost:2181
Connecting to localhost:2181
Welcome to ZooKeeper!
JLine support is disabled

WATCHER::

WatchedEvent state:SyncConnected type:None path:null

ls /
[admin, brokers, cluster, config, consumers, controller, controller_epoch, feature, isr_change_notification, latest_producer_id_block, log_dir_event_notification, zookeeper]
ls -R /
/
/admin
/brokers
/cluster
```

As highlighted above, the cli is connected to the zookeeper running on localhost listening at port 2181.

Let us get all znode:

```
#ls /
```

```

[[zk: localhost:2181(CONNECTED) 1] ls /
[admin, brokers, cluster, config, consumers, controller, controller_epoch, isr_change_notification, latest_producer_id_block, log_dir_event_notification, zookeeper]
[[zk: localhost:2181(CONNECTED) 2] █

```

List all sub nodes created inside the /

```
#ls -R /
```

```
#ls -R /brokers
```

```
#get /brokers/ids/0
```

```

[[zk: localhost:2181(CONNECTED) 22] get /brokers/ids/0
{"listener_security_protocol_map":{"PLAINTEXT":"PLAINTEXT"},"endpoints":["PLAINTEXT://192.168.0.111:9092"],"jmx_port":-1,"host":"192.168.0.111","timestamp":"1597992632471","port":9092,"version":4}
[[zk: localhost:2181(CONNECTED) 23] █

```

Information about the broker which is register in the znode.

Here, the Node 0 is own by the broker IP – 192.168.0.111 with the port 9092.

This way you can determine the Node details whenever there are any issues as shown below:

```

[2020-08-21 12:52:15,093] WARN [Consumer clientId=consumer-console-consumer-26076-1, groupId=console-consumer-26076] Connection to node -1 (localhost/127.0.0.1:9020) could not be established. Broker may not be available. (org.apache.kafka.clients.NetworkClient)
[2020-08-21 12:52:15,093] WARN [Consumer clientId=consumer-console-consumer-26076-1, groupId=console-consumer-26076] Bootstrap broker localhost:9020 (id: -1 rack: null) disconnected (org.apache.kafka.clients.NetworkClient)
^CProcessed a total of 0 messages
(base) Henrys-MacBook-Air:zookeeper henry@notsangham$ █

```

Type quit to exit.

```
#quit
```

Using the Kafka tool `zookeeper-shell.sh` (in Broker Installation folder) we can connect to a ZooKeeper host in our cluster and look at how data gets stored.

```
#cd /opt/kafka/bin
```

```
#sh zookeeper-shell.sh localhost:2181
```

```
#ls /brokers/topics
```

If we look at the path `/brokers/topics` you should see a list of the topics that you have created.

```
[(base) Henrys-MacBook-Air:bin henrypotsangbam$ sh zookeeper-shell.sh localhost:2181
Connecting to localhost:2181
Welcome to ZooKeeper!
JLine support is disabled

WATCHER::

WatchedEvent state:SyncConnected type:None path:null

ls
ls [-s] [-w] [-R] path
ls /brokers/topics
[__consumer_offsets, quickstart-events, test, topic1]
```

You should also be able to see the topic `__consumer_offsets`. That topic is one that you did not create, it is in fact a private topic used internal by Kafka itself. This topic stored the committed offsets for each topic and partition per group id.

The path `/controller` exists in zookeeper and we are running one command to look at that current value

`#get /controller`

```
get /controller  
{"version":1,"brokerid":0,"timestamp":"1688440932193"}
```

Here, broker id = 0 is the controller of this cluster.

Now in the next section, let us configure cluster of zookeeper – 3 nodes.
Ensemble

Setting up a ZooKeeper Ensemble

Create 3 zookeepers' configuration to create zookeeper ensemble of 3 nodes:

The `/opt/zookeeper/conf/zoo1.cfg` file should have the content:

```
dataDir=/opt/data/zookeeper/1
clientPort=2181
initLimit=5
syncLimit=2
server.1=localhost:2888:3888
server.2=localhost:2889:3889
server.3=localhost:2890:3890
```

The `<ZOOKEEPER_HOME>/conf/zoo2.cfg` file should have the content:

```
dataDir=/opt/data/zookeeper/2
clientPort=2182
initLimit=5
syncLimit=2
server.1=localhost:2888:3888
server.2=localhost:2889:3889
server.3=localhost:2890:3890
```

The <ZOOKEEPER_HOME>/conf/zoo3.cfg file should have the content:

```
dataDir=/opt/data/zookeeper/3
clientPort=2183
initLimit=5
syncLimit=2
server.1=localhost:2888:3888
server.2=localhost:2889:3889
server.3=localhost:2890:3890
```

To complete your multi-node configuration, you will specify a node ID on each of the servers. To do this, you will create a `myid` file on each node. Each file will contain a number that correlates to the server number assigned in the configuration file.

Create a data directory for each node

```
#cd /opt/data/zookeeper/
#mkdir 1 2 3
```

Create the `myid` file for each node with its corresponding ID

```
#echo "1" > 1/myid
#echo "2" > 2/myid
#echo "3" > 3/myid
```

```
[(base) Henrys-MacBook-Air:zookeeper henrypotsangbam$ sudo vi 1/myid
[(base) Henrys-MacBook-Air:zookeeper henrypotsangbam$ sudo vi 2/myid
[(base) Henrys-MacBook-Air:zookeeper henrypotsangbam$ sudo vi 3/myid
[(base) Henrys-MacBook-Air:zookeeper henrypotsangbam$ more 1/myid
1
[(base) Henrys-MacBook-Air:zookeeper henrypotsangbam$ pwd
/opt/data/zookeeper
(base) Henrys-MacBook-Air:zookeeper henrypotsangbam$
```

To start the servers, you can simply explicitly reference the configuration files:

```
cd /opt/zookeeper
bin/zkServer.sh start conf/zoo1.cfg
bin/zkServer.sh start conf/zoo2.cfg
bin/zkServer.sh start conf/zoo3.cfg
```

```

4078 QuorumPeerMain
[(base) Henrys-MacBook-Air:zookeeper henrypotsangbam$ sudo bin/zkServer.sh start conf/zoo2.cfg
/usr/bin/java
ZooKeeper JMX enabled by default
Using config: conf/zoo2.cfg
Starting zookeeper ... STARTED
[(base) Henrys-MacBook-Air:zookeeper henrypotsangbam$ sudo bin/zkServer.sh start conf/zoo3.cfg
/usr/bin/java
ZooKeeper JMX enabled by default
Using config: conf/zoo3.cfg
Starting zookeeper ... STARTED
[(base) Henrys-MacBook-Air:zookeeper henrypotsangbam$ jps
4136 Jps
[(base) Henrys-MacBook-Air:zookeeper henrypotsangbam$ sudo jps
4131 QuorumPeerMain
4139 Jps
4107 QuorumPeerMain
4078 QuorumPeerMain
(base) Henrys-MacBook-Air:zookeeper henrypotsangbam$ █

```

You will now start a ZooKeeper command line client and connect to ZooKeeper on **node 1**:

```
#bin/zkCli.sh -server localhost:2181
```

List the newly created znode:

```
#ls /
```

You can verify the leader using command:

```
#cd /opt/zookeeper  
#bin/zkServer.sh status conf/zoo1.cfg  
# bin/zkServer.sh status conf/zoo2.cfg  
# bin/zkServer.sh status conf/zoo3.cfg
```

```
[root@kafka0 zookeeper]# bin/zkServer.sh status conf/zoo2.cfg  
ZooKeeper JMX enabled by default  
Using config: conf/zoo2.cfg  
Client port found: 2182. Client address: localhost.  
Mode: leader  
[root@kafka0 zookeeper]# bin/zkServer.sh status conf/zoo1.cfg  
ZooKeeper JMX enabled by default  
Using config: conf/zoo1.cfg  
Client port found: 2181. Client address: localhost.  
Mode: follower  
[root@kafka0 zookeeper]# bin/zkServer.sh status conf/zoo3.cfg  
ZooKeeper JMX enabled by default  
Using config: conf/zoo3.cfg  
Client port found: 2183. Client address: localhost.  
Mode: follower  
[root@kafka0 zookeeper]# pwd  
/opt/zookeeper
```

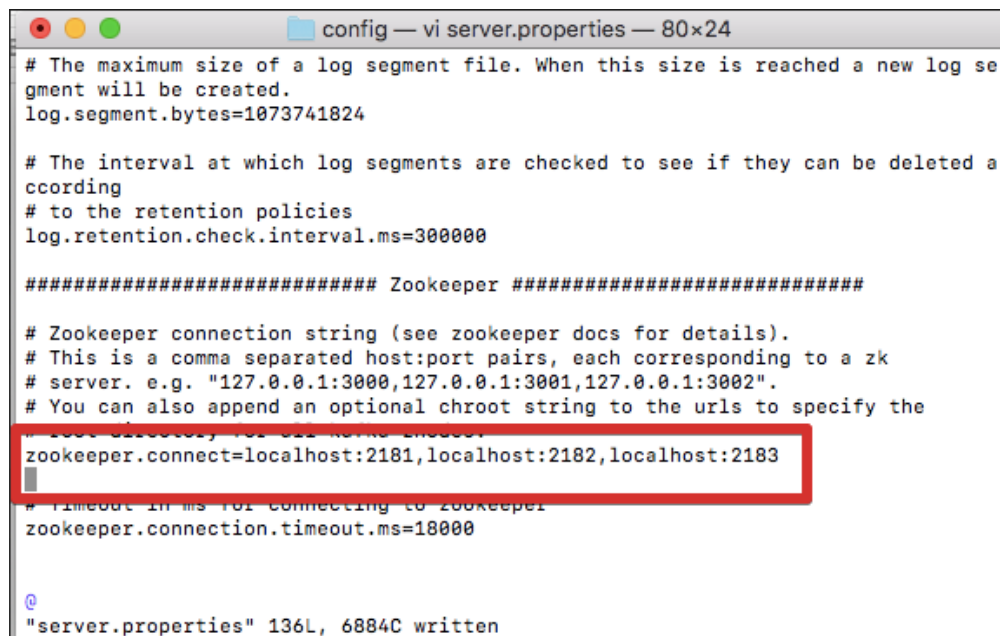
Here, the leader is running in 2182 port.

Finally, let us configure the broker to point to the zookeeper ensemble.

The configuration will be in the Kafka conf folder.

```
#cd /opt/kafka
```

Copy `config/server.properties` into `config/server1.properties` and modify the following two properties in it[server1.properties]. Use cp command.



```
config — vi server.properties — 80x24
# The maximum size of a log segment file. When this size is reached a new log segment will be created.
log.segment.bytes=1073741824

# The interval at which log segments are checked to see if they can be deleted according to the retention policies
log.retention.check.interval.ms=300000

##### Zookeeper #####

# Zookeeper connection string (see zookeeper docs for details).
# This is a comma separated host:port pairs, each corresponding to a zk server. e.g. "127.0.0.1:3000,127.0.0.1:3001,127.0.0.1:3002".
# You can also append an optional chroot string to the urls to specify the root directory for all kafka requests.
zookeeper.connect=localhost:2181,localhost:2182,localhost:2183

# Timeout in ms for connecting to zookeeper
zookeeper.connection.timeout.ms=18000

@
"server.properties" 136L, 6884C written
```

```
log.dirs=/opt/data/kafka-logs1
```

Updated the **Zookeeper.connect** property to connect to all zookeeper nodes.

Start all the zookeepers if not done.

#cd /opt/zookeeper

```
[(base) Henrys-MacBook-Air:zookeeper henrypotsangbam$ sudo bin/zkServer.sh start ]
conf/zoo1.cfg
[Password: ]
/usr/bin/java
ZooKeeper JMX enabled by default
Using config: conf/zoo1.cfg
Starting zookeeper ... STARTED
[(base) Henrys-MacBook-Air:zookeeper henrypotsangbam$ sudo bin/zkServer.sh start ]
conf/zoo2.cfg
/usr/bin/java
ZooKeeper JMX enabled by default
Using config: conf/zoo2.cfg
Starting zookeeper ... STARTED
[(base) Henrys-MacBook-Air:zookeeper henrypotsangbam$ sudo bin/zkServer.sh start ]
conf/zoo3.cfg
/usr/bin/java
ZooKeeper JMX enabled by default
Using config: conf/zoo3.cfg
Starting zookeeper ... STARTED
(base) Henrys-MacBook-Air:zookeeper henrypotsangbam$
```

Start the broker using the modified configuration.

```
#/opt/kafka/bin/kafka-server-start.sh -daemon /opt/kafka/config/server1.properties
```

You can verify from the server.log that the broker is referring to the new property as shown below: `/opt/kafka/logs/server.log`

```
[2020-08-24 08:41:05,479] INFO Registered signal handlers for Ctrl-C, Ctrl-D, Ctrl-K, Ctrl-Q, Ctrl-S, Ctrl-U, Ctrl-Z (org.apache.kafka.common.utils.LoggingSignalHandler)
[2020-08-24 08:41:05,479] INFO starting (kafka.server.KafkaServer)
[2020-08-24 08:41:05,482] INFO Connecting to zookeeper on localhost:2181,localhost:2182,localhost:2183 (kafka.server.KafkaServer)
[2020-08-24 08:41:05,538] INFO [ZooKeeperClient Kafka server] Initializing a new session to localhost:2181,localhost:2182,localhost:2183. (kafka.zookeeper.ZooKeeperClient)
[2020-08-24 08:41:05,564] INFO Client environment:zookeeper.version=3.5.7-f0fdd52973d373ffd9c86b81d99842dc2c7f660e, built on 02/10/2020 11:30 GMT (org.apache.zookeeper.ZooKeeper)


---


[2020-08-24 08:47:42,893] INFO Kafka version: 2.5.0 (org.apache.kafka.common.utils.AppInfoParser)
[2020-08-24 08:47:42,893] INFO Kafka commitId: 66563e712b0b9f84 (org.apache.kafka.common.utils.AppInfoParser)
[2020-08-24 08:47:42,895] INFO Kafka startTimeMs: 1598239062881 (org.apache.kafka.common.utils.AppInfoParser)
[2020-08-24 08:47:42,902] INFO [KafkaServer id=0] started (kafka.server.KafkaServer)
(base) Henrys-MacBook-Air:logs henry$
```

Let us perform some activities using the new set up.

```
# /opt/kafka/bin/kafka-topics.sh --create --bootstrap-server kafka0:9092 --
replication-factor 1 --partitions 2 --topic testz
```

```
# /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka0:9092 --describe --topic testz
```

```
# /opt/kafka/bin/kafka-console-producer.sh --broker-list kafka0:9092 --topic testz
    Test Message 1
    Test Message 2
To exit : ^D
```

Consume the above messages:

```
# /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server kafka0:9092 --topic
testz --from-beginning
```

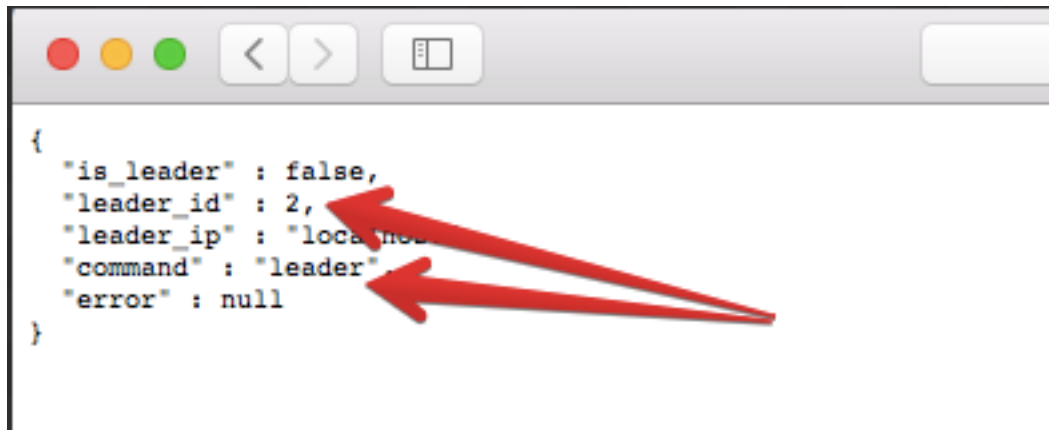
```
[(base) Henrys-MacBook-Air:logs henrypotsangbam$ sudo /opt/kafka/bin/kafka-console-producer.sh --broker-list localh
ost:9092 --topic testz
>Test Message 1
Test Message 2
[>>(base) Henrys-MacBook-Air:logs henrypotsangbam$ sudo /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server
ocalhost:9092 --topic testz --from-beginning
Test Message 1
Test Message 2
```

Let us verify the Leader of the zookeeper and Kill the leader process.

```
#curl http://localhost:8080/commands/leader
```

or CLI :

```
#/opt/zookeeper/bin/zkServer.sh status /opt/zookeeper/conf/zoo2.cfg
```



```
[root@kafka0 /]# /opt/zookeeper/bin/zkServer.sh status /opt/zookeeper/conf/zoo2.cfg
ZooKeeper JMX enabled by default
Using config: /opt/zookeeper/conf/zoo2.cfg
Client port found: 2182. Client address: localhost.
Mode: leader
[root@kafka0 /]#
```

You can verify the other configuration if 2 is not the leader.

Since 2 is the leader. Let us kill the process Id of the Zookeeper id 2.

```
#ps -eaf | grep zoo2.cfg
```

```

(base) Henrys-MacBook-Air:config henrypotsangbam$ sudo ps -eaf | grep zoo2.cfg
501 3725 537 0 9:05AM ttys000 0:00.00 grep zoo2.cfg
0 684 1 0 8:39AM ttys001 0:05.91 /usr/bin/java -Dzookeeper.log.dir=/opt/zookeeper/bin/../logs -Dzookeeper.log.fil
e=zookeeper-root-server-Henrys-MacBook-Air.local.log -Dzookeeper.root.logger=INFO,CONSOLE -XX:+HeapDumpOnOutOfMemoryError -XX:OnO
utOfMemoryError=kill -9 %p -cp /opt/zookeeper/bin/../zookeeper-server/target/classes:/opt/zookeeper/bin/../build/classes:/opt/zoo
keeper/bin/../zookeeper-server/target/lib/*.jar:/opt/zookeeper/bin/../build/lib/*.jar:/opt/zookeeper/bin/../lib/zookeeper-prometh
eus-metrics-3.6.1.jar:/opt/zookeeper/bin/../lib/zookeeper-jute-3.6.1.jar:/opt/zookeeper/bin/../lib/zookeeper-3.6.1.jar:/opt/zooke
eper/bin/../lib/snappy-java-1.1.7.jar:/opt/zookeeper/bin/../lib/slf4j-log4j12-1.7.25.jar:/opt/zookeeper/bin/../lib/slf4j-api-1.7.
25.jar:/opt/zookeeper/bin/../lib/simpleclient_servlet-0.6.0.jar:/opt/zookeeper/bin/../lib/simpleclient_hotspot-0.6.0.jar:/opt/zoo
keeper/bin/../lib/simpleclient_common-0.6.0.jar:/opt/zookeeper/bin/../lib/simpleclient-0.6.0.jar:/opt/zookeeper/bin/../lib/netty-
transport-native-unix-common-4.1.48.Final.jar:/opt/zookeeper/bin/../lib/netty-transport-native-epoll-4.1.48.Final.jar:/opt/zookee
per/bin/../lib/netty-transport-4.1.48.Final.jar:/opt/zookeeper/bin/../lib/netty-resolver-4.1.48.Final.jar:/opt/zookeeper/bin/../l
ib/netty-handler-4.1.48.Final.jar:/opt/zookeeper/bin/../lib/netty-common-4.1.48.Final.jar:/opt/zookeeper/bin/../lib/netty-codec-4
.1.48.Final.jar:/opt/zookeeper/bin/../lib/netty-buffer-4.1.48.Final.jar:/opt/zookeeper/bin/../lib/metrics-core-3.2.5.jar:/opt/zoo
keeper/bin/../lib/log4j-1.2.17.jar:/opt/zookeeper/bin/../lib/json-simple-1.1.1.jar:/opt/zookeeper/bin/../lib/jline-2.11.jar:/opt/
zookeeper/bin/../lib/jetty-util-9.4.24.v20191120.jar:/opt/zookeeper/bin/../lib/jetty-servlet-9.4.24.v20191120.jar:/opt/zookeeper/
bin/../lib/jetty-server-9.4.24.v20191120.jar:/opt/zookeeper/bin/../lib/jetty-security-9.4.24.v20191120.jar:/opt/zookeeper/bin/../
lib/jetty-io-9.4.24.v20191120.jar:/opt/zookeeper/bin/../lib/jetty-http-9.4.24.v20191120.jar:/opt/zookeeper/bin/../lib/javax.servl
et-api-3.1.0.jar:/opt/zookeeper/bin/../lib/jackson-databind-2.10.3.jar:/opt/zookeeper/bin/../lib/jackson-core-2.10.3.jar:/opt/zoo
keeper/bin/../lib/jackson-annotations-2.10.3.jar:/opt/zookeeper/bin/../lib/commons-lang-2.6.jar:/opt/zookeeper/bin/../lib/commons
-cli-1.2.jar:/opt/zookeeper/bin/../lib/audience-annotations-0.5.0.jar:/opt/zookeeper/bin/../lib/zoo-common-*.jar:/opt/zookeeper/bin/..
/zookeeper-server/src/main/resources/lib/*.jar:/opt/zookeeper/bin/../conf: -Xmx1000m -Dcom.sun.management.jmxremote -Dcom.sun.man
agement.jmxremote.local.only=false org.apache.zookeeper.server.quorum.QuorumPeerMain -conf/zoo2.cfg
(base) Henrys-MacBook-Air:config henrypotsangbam$

```

In this case, the process id is 684.

kill -9 684

```

(base) Henrys-MacBook-Air:config henrypotsangbam$ sudo kill -9 684
(base) Henrys-MacBook-Air:config henrypotsangbam$ sudo jps
3745 Jps
659 QuorumPeerMain
3382 ConsoleConsumer
710 QuorumPeerMain
2026 Kafka
(base) Henrys-MacBook-Air:config henrypotsangbam$

```

You can refresh the browser to determine the new leader.

```
#curl http://localhost:8080/commands/leader
```

```
{  
  "is_leader" : false,  
  "leader_id" : 3,  
  "leader_ip" : "localhost",  
  "command" : "leader",  
  "error" : null  
}
```

or using cli.

```
[root@kafka0 /]# /opt/zookeeper/bin/zkServer.sh status /opt/zookeeper/conf/zoo2.cfg  
ZooKeeper JMX enabled by default  
Using config: /opt/zookeeper/conf/zoo2.cfg  
Client port found: 2182. Client address: localhost.  
Error contacting service. It is probably not running.  
[root@kafka0 /]# /opt/zookeeper/bin/zkServer.sh status /opt/zookeeper/conf/zoo3.cfg  
ZooKeeper JMX enabled by default  
Using config: /opt/zookeeper/conf/zoo3.cfg  
Client port found: 2183. Client address: localhost.  
Mode: leader
```

So which node is the leader in your case? Here its 3

Try sending some messages again:

```
((base) Henrys-MacBook-Air:logs henrypotsangbam$ sudo /opt/kafka/bin/kafka-console-producer.sh --broker-list localhost:9092 --topic testz
[Password:
]>Another Message
>
^CProcessed a total of 0 messages
(base) Henrys-MacBook-Air:config henrypotsangbam$ sudo /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic testz --from-beginning
Test Message 1
Test Message 2
Another Message
█
```

You should be able to consume messages without any issue.

Get some config details from the zookeeper: On ZK cli

```
get /controller
get /brokers/topics/testz
get /brokers/ids/0
get /zookeeper/config
```

```

[[zk: localhost:2181(CONNECTED) 19] get /controller
{"version":1,"brokerid":0,"timestamp":"1598239062599"}
[[zk: localhost:2181(CONNECTED) 20] get /brokers/topics/testz
{"version":2,"partitions":{"1":[0],"0":[0]},"adding_replicas":{"},"removing_replicas":{"}}
[[zk: localhost:2181(CONNECTED) 21] get /brokers/ids/0
{"listener_security_protocol_map":{"PLAINTEXT":"PLAINTEXT"},"endpoints":["PLAINTEXT://quickstart.cloudera:9092"],"jmx_port":-1,"host":"quickstart.cloudera","timestamp":"1598239062331","port":9092,"version":4}
[[zk: localhost:2181(CONNECTED) 22] get /zookeeper/config
server.1=localhost:2888:3888:participant
server.2=localhost:2889:3889:participant
server.3=localhost:2890:3890:participant
version=0
[[zk: localhost:2181(CONNECTED) 23] █

```

You can shutdown the zookeeper and the broker.
Close the consumer and producer console too.

`/opt/kafka/bin/kafka-server-stop.sh`
`/opt/kafka/bin/zookeeper-server-stop.sh`

----- lab Ends Here -----

Log:

server.1=localhost:2888:3888 (2181)
server.2=localhost:2889:3889. (2182) - First Leader
server.3=localhost:2890:3890 -(2183) Second Leader

Scenarios : 2 was leader , after 2 shutdown -> 3 Became a leader

7. Kafka 3 Nodes - cluster – 90 Minutes

In this tutorial, we are going to run many Kafka Nodes on our system to Understand Kafka Failover in a cluster environment. So, you will need at least 8 GB of RAM in your system. You can run just two servers if you have less memory than 8 GB.

We are going to create a replicated topic and then demonstrate consumer group coordination along with broker failover. We will also demonstrate load balancing of Kafka consumers. We show how, with many groups, Kafka acts like a Publish/Subscribe. But, when we put all of our consumers in the same group, Kafka will load share the messages to the consumers in the same group (more like a queue than a topic in a traditional MOM sense).

If not already running, start **ZooKeeper**.
Also, shut down Kafka from the first tutorial.

Next, you need to copy server properties for three brokers (detailed instructions to follow). Then we will modify these Kafka server properties to add unique Kafka ports, Kafka log locations, and unique Broker ids.
Then we will create three scripts to start these servers up using these properties, and then start the servers.
Lastly, we create replicated topic and use it to demonstrate Kafka consumer failover, and Kafka broker failover.

Create three new Kafka server-n.properties files

In this section, we will perform the following steps:

- copy the existing Kafka `server.properties` to `server-0.properties`, `server-1.properties`, and `server-2.properties`.
- Update `server-0.properties` to set `log.dirs` to `"/opt/data/kafka-logs/kafka-0"`.
- Modify `server-1.properties` to set `port` to `9093`, broker `id` to `1`, and `log.dirs` to `"/opt/data/kafka-logs/kafka-1"`.
- Lastly modify `server-2.properties` to use `port` `9094`, broker `id` `2`, and `log.dirs` `"/opt/data/kafka-logs/kafka-2"`.

Copy server properties file as follows: We will store all server's configuration in a single folder config - `kafka-config/config`.

```
$ cd /opt
$ mkdir -p kafka-config/config
$ cp kafka/config/server.properties kafka-config/config/server-0.properties
$ cp kafka/config/server.properties kafka-config/config/server-1.properties
$ cp kafka/config/server.properties kafka-config/config/server-2.properties
```

```
[root@tos opt]# mkdir -p kafka-config/config
[root@tos opt]# pwd
/opt
[root@tos opt]# ls
couchbase  data  jdk1.8.0_45  kafka  kafka-config  zookeeper  zookeeper.out
[root@tos opt]# cp kafka/config/server.properties kafka-config/server-0.properties
[root@tos opt]# ls
couchbase  data  jdk1.8.0_45  kafka  kafka-config  zookeeper  zookeeper.out
[root@tos opt]# cp kafka/config/server.properties kafka-config/config/server-1.properties
[root@tos opt]# cp kafka/config/server.properties kafka-config/config/server-2.properties
[root@tos opt]#
```

With your favourite text editor update server-0.properties so that **log.dirs** is set to **./logs/kafka-0**. Leave the rest of the file as it is. Make sure **log.dirs** is only defined once.

```
#vi /opt/kafka-config/config/server-0.properties
broker.id=0
listeners=PLAINTEXT://kafka0:9092
advertised.listeners=PLAINTEXT://kafka0:9092
log.dirs=/opt/data/kafka-logs/kafka-0
```

With your favorite text editor change `log.dirs`, `broker.id` and `log.dirs` of `server-1.properties` as follows.

```
#vi /opt/kafka-config/config/server-1.properties
broker.id=1
listeners=PLAINTEXT://kafka0:9093
advertised.listeners=PLAINTEXT://kafka0:9093
log.dirs=/opt/data/kafka-logs/kafka-1
```

With your favorite text editor change `log.dirs`, `broker.id` and `log.dirs` of `server-2.properties` as follows.

```
#vi /opt/kafka-config/config/server-2.properties
broker.id=2
listeners=PLAINTEXT://kafka0:9094
advertised.listeners=PLAINTEXT://kafka0:9094
log.dirs=/opt/data/kafka-logs/kafka-2
```

Create Start-up scripts for these three Kafka servers

The startup scripts will just run `kafka-server-start.sh` with the corresponding properties file.

```
#vi /opt/scripts/startB0.sh
```

Copy the following in the above file.

```
#!/usr/bin/env bash
## Run Kafka
/opt/kafka/bin/kafka-server-start.sh "/opt/kafka-config/config/server-
0.properties"
```

Do the same for the remaining 2 nodes.

```
#vi /opt/scripts/startB1.sh
#!/usr/bin/env bash
## Run Kafka
/opt/kafka/bin/kafka-server-start.sh "/opt/kafka-config/config/server-
1.properties"
```

```
#vi /opt/scripts/startB2.sh
#!/usr/bin/env bash
## Run Kafka
/opt/kafka/bin/kafka-server-start.sh "/opt/kafka-config/config/server-
2.properties"
```

Notice we are passing the Kafka server properties files that we created in the last step. Now run all three in separate terminals/shells.

Run Kafka servers each in separate terminal from /opt/kafka-config

```
#cd /opt/scripts
// to make the scripts executable.
#chmod 755 startB0.sh startB1.sh startB2.sh
$ ./startB0.sh
$ ./startB1.sh
$ ./startB2.sh
```

Give these servers a couple of minutes to startup and connect to ZooKeeper from a separate terminal.

The image shows three terminal windows running Kafka. The top-left window shows the startup of broker 2, including GroupCoordinator, GroupMetadataManager, and TransactionCoordinator components. The top-right window shows the startup of broker 1, including Log, GroupCoordinator, and TransactionCoordinator components. The bottom window shows the startup of broker 1, including TransactionMarkerChannelManager, GroupCoordinator, and TransactionCoordinator components. All windows show the Kafka version 1.1.0 and the commitId fdcf75ea326b8e07.

```

root@tos:/opt/kafka-config
[2018-05-16 15:17:50,404] INFO [GroupCoordinator 2]: Starting up. (kafka.coordin
ator.group.GroupCoordinator)
[2018-05-16 15:17:50,411] INFO [GroupCoordinator 2]: Startup complete. (kafka.co
ordinator.group.GroupCoordinator)
[2018-05-16 15:17:50,564] INFO [GroupMetadataManager brokerId=2] Removed 0 expir
ed offsets in 140 milliseconds. (kafka.coordinator.group.GroupMetadataManager)
[2018-05-16 15:17:50,637] INFO [ProducerId Manager 2]: Acquired new producerId b
lock (brokerId=2,blockStartProducerId:4000,blockEndProducerId:4999) by writing t
o Zk with path version 5 (kafka.coordinator.transaction.ProducerIdManager)
[2018-05-16 15:17:50,777] INFO [TransactionCoordinator id=2] Starting up. (kafka
.coordinator.transaction.TransactionCoordinator)
[2018-05-16 15:17:50,788] INFO [TransactionCoordinator id=2] Startup complete. (
kafka.coordinator.transaction.TransactionCoordinator)
[2018-05-16 15:17:51,303] INFO [Transaction Marker Channel Manager 2]: Starting
(kafka.coordinator.transaction.TransactionMarkerChannelManager)
[2018-05-16 15:17:51,574] INFO [/config/changes-event-process-thread]: Starting
(kafka.common.ZkNodeChangeNotificationListener$ChangeEventProcessThread)
[2018-05-16 15:17:51,723] INFO Kafka version : 1.1.0 (org.apache.kafka.common.ut
ils.AppInfoParser)
[2018-05-16 15:17:51,723] INFO Kafka commitId : fdcf75ea326b8e07 (org.apache.kaf
ka.common.utils.AppInfoParser)
[2018-05-16 15:17:51,728] INFO [KafkaServer id=2] started (kafka.server.KafkaSer
ver)

@tos:/opt/kafka-config
[2018-05-16 15:09:28,919] INFO [Log partition=test-0, dir=/opt/data/kafka-logs/k
] Completed load of log with 1 segments, log start offset 0 and log end of
in 154 ms (kafka.log.Log)
[2018-05-16 15:09:28,940] INFO Created log for partition test-0 in /opt/data/kaf
s/kafka-0 with properties (compression.type -> producer, message.format.ver
-> 1.1-IV0, file.delete.delay.ms -> 60000, max.message.bytes -> 1000012, m
action.lag.ms -> 0, message.timestamp.type -> CreateTime, min.insync.repl
-> 1, segment.jitter.ms -> 0, preallocate -> false, min.cleanable.dirty.rat
0.5, index.interval.bytes -> 4096, unclean.leader.election.enable -> false
on.ms -> 86400000, cleanup.policy -> [del
segment.ms -> 604800000, segment.bytes ->
message.timestamp.difference.max.ms -> 9
-> 10485760, flush.messages -> 922337203
on test-0 broker=0] No checkpointed highw
(kafka.cluster.Partition)
loaded for partition test-0 with initial
)
on test-0 broker=0] test-0 starts at Lead
er Epoch was: -1 (kafka.cluster.Partition
AlterLogDirsManager on broker 0] Added fe
er.ReplicaAlterLogDirsManager)

@tos:/opt/kafka-config
[2018-05-16 15:15:08,676] INFO [Transaction Marker Channel Manager 1]: Starting
(kafka.coordinator.transaction.TransactionMarkerChannelManager)
[2018-05-16 15:15:09,267] INFO [/config/changes-event-process-thread]: Starting
(kafka.common.ZkNodeChangeNotificationListener$ChangeEventProcessThread)
[2018-05-16 15:15:09,281] INFO Kafka version : 1.1.0 (org.apache.kafka.common.ut
ils.AppInfoParser)
[2018-05-16 15:15:09,282] INFO Kafka commitId : fdcf75ea326b8e07 (org.apache.kaf
ka.common.utils.AppInfoParser)
[2018-05-16 15:15:09,285] INFO [KafkaServer id=1] started (kafka.server.KafkaSer
ver)

```

You should have as many terminals as shown above at this point.

Create a replicated topic **my-failsafe-topic**

Now we will create a replicated topic that the console producers and console consumers can use.

Open a separate terminal and execute the following;

```
#/opt/kafka/bin/kafka-topics.sh --create --bootstrap-server kafka0:9092 --  
replication-factor 3 --partitions 13 --topic my-failsafe-topic
```

```
[root@kafka0 /]# /opt/kafka/bin/kafka-topics.sh --create --bootstrap-server kafka0:9092 --replication-factor 3 --partitions 13 --topic m  
y-failsafe-topic  
Created topic my-failsafe-topic.  
[root@kafka0 /]#
```

Notice that the replication factor gets set to 3, and the topic name is **my-failsafe-topic**, and like before it has 13 partitions.

Start Kafka Consumer that uses Replicated Topic

Start consumer with the following script in a separate terminal;

```
#/opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server  
kafka0:9094,kafka0:9092 --topic my-failsafe-topic --from-beginning
```

```
[root@kafka0 /]# /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server kafka0:9094,kafka0:9092 --topic my-failsafe-topic --from-beginning  
█
```

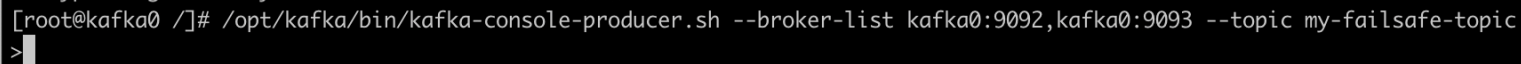
Notice that a list of Kafka servers is passed to **--bootstrap-server** parameter. Only, two of the three servers get passed that we ran earlier. Even though only one broker is needed, the

consumer client will learn about the other broker from just one server. Usually, you list multiple brokers in case there is an outage so that the client can connect.

Start Kafka Producer that uses Replicated Topic

Next, we create a script that starts the producer. Then launch a producer console with the script you create.

```
#/opt/kafka/bin/kafka-console-producer.sh --broker-list kafka0:9092,kafka0:9093 --  
topic my-failsafe-topic
```



```
[root@kafka0 /]# /opt/kafka/bin/kafka-console-producer.sh --broker-list kafka0:9092,kafka0:9093 --topic my-failsafe-topic  
>
```

Notice we start Kafka producer and pass it a list of Kafka Brokers to use via the parameter **--broker-list**.

Now use the start producer script to launch the producer as follows.

Now send messages

Now send some message from the producer to Kafka and see those messages consumed by the consumer.

Producer Console.

```
[root@kafka0 /]# /opt/kafka/bin/kafka-console-producer.sh --broker-list kafka0:9092,kafka0:9093 --topic my-failsafe-topic
>How are you, kafka
>Its' working
>
```

Consumer Console

```
[root@kafka0 /]# /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server kafka0:9094,kafka0:9092 --topic my-failsafe-topic --from-beginning
How are you, kafka
Its' working
```

Now Start two more consumers and send more messages.

Now Start two more consumers in their own terminal window and send more messages from the producer. (Replace the hostname of your server aka localhost)

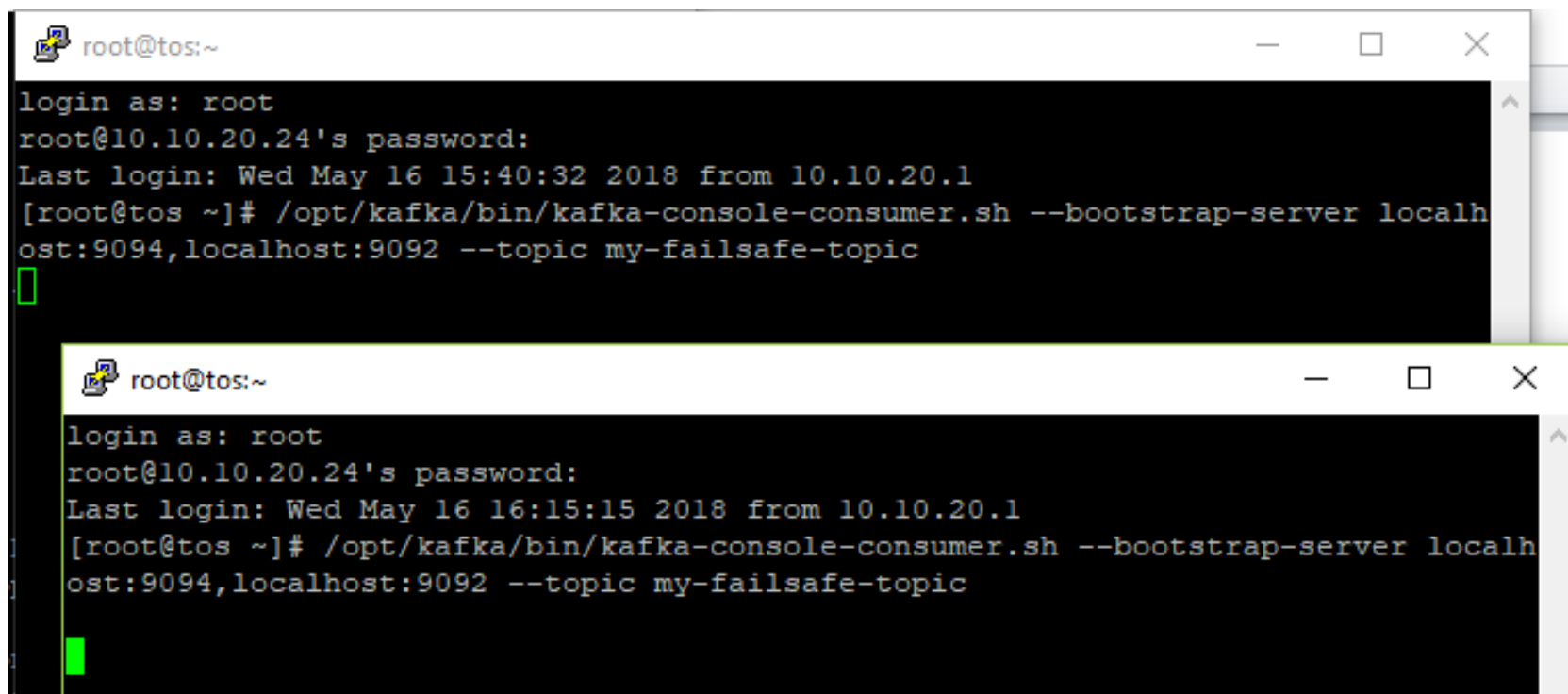
Consumer 1.

```
#/opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server
kafka0:9094,kafka0:9092 --topic my-failsafe-topic --from-beginning
```

Consumer 2.

```
#/opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server  
kafka0:9094,kafka0:9092 --topic my-failsafe-topic --from-beginning
```

The two consumer consoles will be as shown below



The image shows two terminal windows, one above the other, both running on a system named 'tos' as the 'root' user. Each window displays the login process and the execution of the Kafka console consumer command. The top window shows a login at 15:40:32 on May 16, 2018, from IP 10.10.20.1. The bottom window shows a login at 16:15:15 on the same date and from the same IP. Both windows have a green cursor at the end of the command line, indicating they are ready for input.

```
root@tos:~  
login as: root  
root@10.10.20.24's password:  
Last login: Wed May 16 15:40:32 2018 from 10.10.20.1  
[root@tos ~]# /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localh  
ost:9094,localhost:9092 --topic my-failsafe-topic  
█
```

```
root@tos:~  
login as: root  
root@10.10.20.24's password:  
Last login: Wed May 16 16:15:15 2018 from 10.10.20.1  
[root@tos ~]# /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localh  
ost:9094,localhost:9092 --topic my-failsafe-topic  
█
```

Producer Console will be as shown below:

```
login as: root
root@10.10.20.24's password:
Last login: Wed May 16 15:20:40 2018 from 10.10.20.1
[root@tos ~]# /opt/kafka/bin/kafka-console-producer.sh --broker-list localhos
092,localhost:9093 --topic my-failsafe-topic
>How are you?
>Great Kafka is working
>message 1
>hope every one is getting this mesage
>Very Good
>
```

Consumer Console 1st, you should be able to view the messages whatever you type on the producer console after the new consumer console was started.

```
login as: root
root@10.10.20.24's password:
Last login: Wed May 16 15:40:32 2018 from 10.10.20.1
[root@tos ~]# /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:9094,localhost:9092 --topic my-failsafe-topic
message 1
hope every one is getting this message
Very Good
```

Consumer Console 2nd in new Terminal, similarly all the messages that were type on the producer console should be also display in the second console after it was started.
Consumer Console 2nd in new Terminal

```
created topic 'my-failsafe-topic'
[root@tos ~]# /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:9094,localhost:9092 --topic my-failsafe-topic
How are you?
Great Kafka is working
message 1
hope every one is getting this message
Very Good
```

Notice that the messages are sent to all of the consumers because each consumer is in a different consumer group.

Change consumer to be in their own consumer group.

Stop the producers and the consumers before, but leave Kafka and ZooKeeper running. You can use `ctrl + c`.

We want to put all of the consumers in same *consumer group*. This way the consumers will share the messages as each consumer in the *consumer group* will get its share of partitions.

Run the following scripts three times – from different console.

```
#/opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server  
kafka0:9094,kafka0:9092 --topic my-failsafe-topic --consumer-property  
group.id=mygroup
```

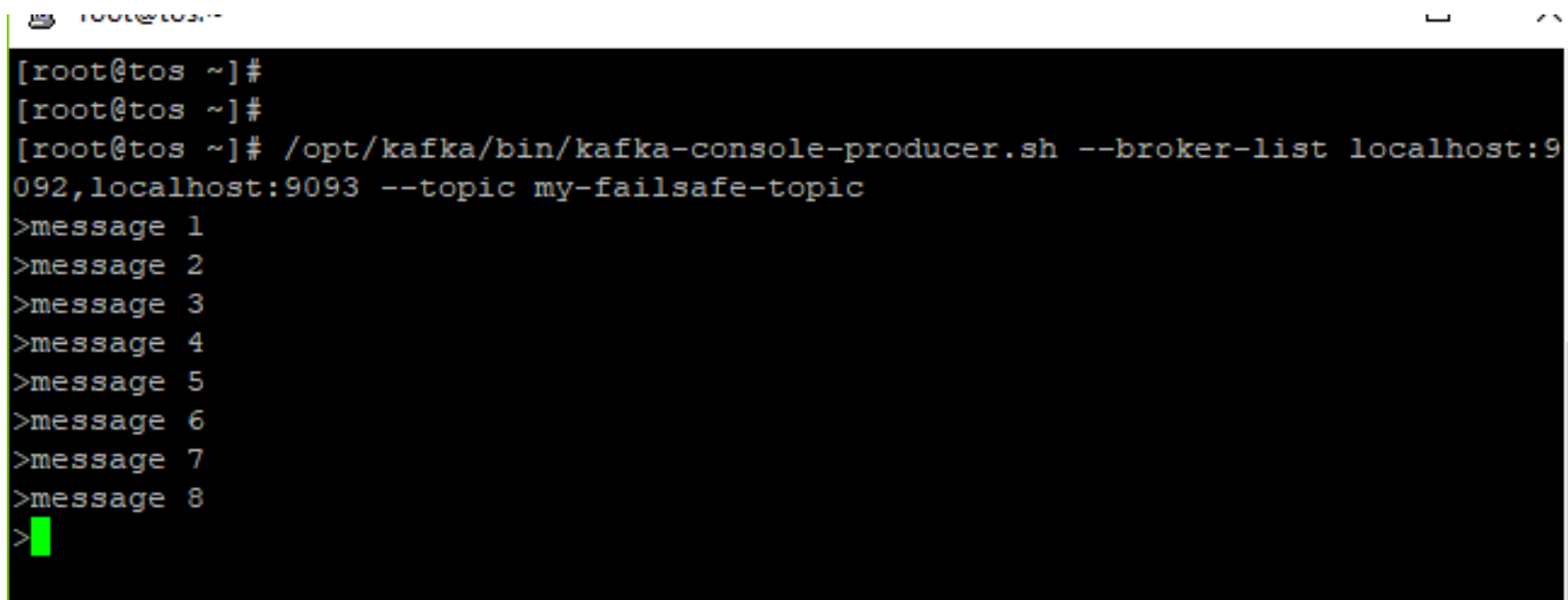
Notice that the script is the same as before except we added `--consumer-property group.id=mygroup` which will put every consumer that runs with this script into the `mygroup` consumer group.

Now we just run the producer and three consumers.

Run Producer Console

```
/opt/kafka/bin/kafka-console-producer.sh --broker-list kafka0:9092,kafka0:9093 --  
topic my-failsafe-topic
```

Now send eight messages from the Kafka producer console.
Producer Console

A terminal window with a black background and white text. The prompt is [root@tos ~]#. The command entered is /opt/kafka/bin/kafka-console-producer.sh --broker-list localhost:9092,localhost:9093 --topic my-failsafe-topic. The output shows eight messages being sent, each preceded by >message. The messages are numbered 1 through 8. The prompt > is followed by a green cursor.

```
[root@tos ~]#  
[root@tos ~]#  
[root@tos ~]# /opt/kafka/bin/kafka-console-producer.sh --broker-list localhost:9092,localhost:9093 --topic my-failsafe-topic  
>message 1  
>message 2  
>message 3  
>message 4  
>message 5  
>message 6  
>message 7  
>message 8  
>
```

Notice that the messages are spread evenly among the consumers. If there is no Key then it may receive on only consumer console.

Sample Output:

```
[root@kafka0 /]# /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server  
kafka0:9094,kafka0:9092 --topic my-failsafe-topic --consumer-property group.id=mygroup  
m1  
m2
```

m3
m4
m5
m6
m7
m8

Kafka Consumer Failover

Next, let's demonstrate consumer failover by killing one of the consumers and sending seven more messages. Kafka should divide up the work to the consumers that are running. First, kill the third consumer (CTRL-C in the consumer terminal does the trick).in which all message get consumed.

```
[root@tos ~]# /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server local
host:9094,localhost:9092 --topic my-failsafe-topic --consumer-property group.
id=mygroup
message 1
message 3
^CProcessed a total of 2 messages
[root@tos ~]#
```

Now send seven more messages from the Kafka console-producer.

Producer Console - send seven more messages m8 through m14

```
[root@tos ~]# /opt/kafka/bin/kafka-console-producer.sh --broker-list localhost:9092,localhost:9093 --topic my-failsafe-topic
>message 1
>message 2
>message 3
>message 4
>message 5
>message 6
>message 7
>message 8
>
>message 9
>m 10
>m11
>m 12
>m 13
>m 14
>m 15
>
```

Notice that the messages are consumed in one of the remaining consumers.

```
[root@kafka0 /]# /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server kafka0:9094,kafka0:9092 --topic my-failsafe-topic --consumer-property group.id=mygroup
m9
m10
m11
m12
m13
```

We killed one consumer, sent seven more messages, and saw Kafka sent it to one of the remaining consumers. ***Kafka consumer failover works!***

Create Kafka Describe Topic Script

You can use `kafka-topics.sh` to see how the Kafka topic is laid out among the Kafka brokers. The `---describe` will show partitions, ISRs, and broker partition leadership.

```
#/opt/kafka/bin/kafka-topics.sh --describe --topic my-failsafe-topic --bootstrap-server kafka0:9092
```

```
[root@tos ~]# /opt/kafka/bin/kafka-topics.sh --describe --topic my-failsafe-topic --zookeeper localhost:2181
Topic:my-failsafe-topic PartitionCount:13      ReplicationFactor:3      Configs:
  Topic: my-failsafe-topic      Partition: 0      Leader: 2      Replicas: 2,0,1 Isr: 0,1,2
  Topic: my-failsafe-topic      Partition: 1      Leader: 0      Replicas: 0,1,2 Isr: 0,1,2
  Topic: my-failsafe-topic      Partition: 2      Leader: 1      Replicas: 1,2,0 Isr: 0,1,2
  Topic: my-failsafe-topic      Partition: 3      Leader: 2      Replicas: 2,1,0 Isr: 0,1,2
  Topic: my-failsafe-topic      Partition: 4      Leader: 0      Replicas: 0,2,1 Isr: 0,1,2
  Topic: my-failsafe-topic      Partition: 5      Leader: 1      Replicas: 1,0,2 Isr: 0,1,2
  Topic: my-failsafe-topic      Partition: 6      Leader: 2      Replicas: 2,0,1 Isr: 0,1,2
  Topic: my-failsafe-topic      Partition: 7      Leader: 0      Replicas: 0,1,2 Isr: 0,1,2
  Topic: my-failsafe-topic      Partition: 8      Leader: 1      Replicas: 1,2,0 Isr: 0,1,2
  Topic: my-failsafe-topic      Partition: 9      Leader: 2      Replicas: 2,1,0 Isr: 0,1,2
  Topic: my-failsafe-topic      Partition: 10     Leader: 0      Replicas: 0,2,1 Isr: 0,1,2
  Topic: my-failsafe-topic      Partition: 11     Leader: 1      Replicas: 1,0,2 Isr: 0,1,2
  Topic: my-failsafe-topic      Partition: 12     Leader: 2      Replicas: 2,0,1 Isr: 0,1,2
[root@tos ~]#
```

Run describe-topics

We are going to lists which broker owns (leader of) which partition, and list replicas and ISRs of each partition. ISRs are replicas that are up to date. Remember there are 13 topics.

Topology of Kafka Topic Partition Ownership

Notice how each broker gets a share of the partitions as leaders and followers. Also, see how Kafka replicates the partitions on each broker.

Test Broker Failover by killing 1st server

Let's kill the first broker, and then test the failover.

Kill the first broker

```
$ kill `ps aux | grep java | grep server-0.properties | tr -s " " | cut -d " " -f2`
```

```
[2018-05-17 17:38:07,157] INFO [ThrottledRequestReaper-Request]: Shutdown completed (kafka.server.ClientQuotaManager$ThrottledRequestReaper)
[2018-05-17 17:38:07,158] INFO [SocketServer brokerId=0] Shutting down socket server (kafka.network.SocketServer)
[2018-05-17 17:38:07,267] INFO [SocketServer brokerId=0] Shutdown completed (kafka.network.SocketServer)
[2018-05-17 17:38:07,286] INFO [KafkaServer id=0] shut down completed (kafka.server.KafkaServer)
[root@tos kafka-config]#
```

You can stop the first broker by hitting CTRL-C in the broker terminal or by running the above command.

Now that the first Kafka broker has stopped, let's use Kafka `topics describe` to see that new leaders were elected!

Run `describe-topics` again to see leadership change

```
#/opt/kafka/bin/kafka-topics.sh --describe --topic my-failsafe-topic --bootstrap-server kafka0:9093
```

```
[root@kafka0 /]# /opt/kafka/bin/kafka-topics.sh --describe --topic my-failsafe-topic --bootstrap-server kafka0:9093
Topic: my-failsafe-topic      TopicId: zsM1HqfoRciCUX7qwLBM3w  PartitionCount: 13 ReplicationFactor: 3  Configs:
Topic: my-failsafe-topic      Partition: 0      Leader: 1  Replicas: 0,1,2  Isr: 1,2      Elr: N/A      LastKnownElr: N/A
Topic: my-failsafe-topic      Partition: 1      Leader: 2  Replicas: 2,0,1  Isr: 2,1      Elr: N/A      LastKnownElr: N/A
Topic: my-failsafe-topic      Partition: 2      Leader: 1  Replicas: 1,2,0  Isr: 1,2      Elr: N/A      LastKnownElr: N/A
Topic: my-failsafe-topic      Partition: 3      Leader: 2  Replicas: 0,2,1  Isr: 2,1      Elr: N/A      LastKnownElr: N/A
Topic: my-failsafe-topic      Partition: 4      Leader: 2  Replicas: 2,1,0  Isr: 2,1      Elr: N/A      LastKnownElr: N/A
Topic: my-failsafe-topic      Partition: 5      Leader: 1  Replicas: 1,0,2  Isr: 1,2      Elr: N/A      LastKnownElr: N/A
Topic: my-failsafe-topic      Partition: 6      Leader: 1  Replicas: 0,1,2  Isr: 1,2      Elr: N/A      LastKnownElr: N/A
Topic: my-failsafe-topic      Partition: 7      Leader: 2  Replicas: 2,0,1  Isr: 2,1      Elr: N/A      LastKnownElr: N/A
Topic: my-failsafe-topic      Partition: 8      Leader: 1  Replicas: 1,2,0  Isr: 1,2      Elr: N/A      LastKnownElr: N/A
Topic: my-failsafe-topic      Partition: 9      Leader: 2  Replicas: 0,2,1  Isr: 2,1      Elr: N/A      LastKnownElr: N/A
Topic: my-failsafe-topic      Partition: 10     Leader: 2  Replicas: 2,1,0  Isr: 2,1      Elr: N/A      LastKnownElr: N/A
Topic: my-failsafe-topic      Partition: 11     Leader: 1  Replicas: 1,0,2  Isr: 1,2      Elr: N/A      LastKnownElr: N/A
Topic: my-failsafe-topic      Partition: 12     Leader: 1  Replicas: 0,1,2  Isr: 1,2      Elr: N/A      LastKnownElr: N/A
[root@kafka0 /]#
```

Notice how Kafka spreads the leadership over the 2nd and 3rd Kafka brokers.

Stop all the server nodes and the zookeeper.

```
/opt/kafka/bin/kafka-server-stop.sh  
/opt/kafka/bin/zookeeper-server-stop.sh
```

Close the producers and consumer console.

Ctrl+C

----- Lab Ends here -----

8. Securing Kafka SSL – 90 Minutes

Lab Objectives

In this lab, we will be working with a **single-node Kafka broker** to configure **SSL-based encryption** and **authentication**. Our goal is to enable secure communication between the Kafka client and the broker through SSL, and to prepare for the implementation of **Access Control Lists (ACLs)**, which will be addressed in a subsequent lab. You will generate a self-signed Certificate Authority, use it to sign the broker's identity, and force clients to authenticate via SSL before they can produce or consume data.

Prerequisites:

- **Backup Configuration:** Before making any changes to the server configuration, it's essential to back up your `server.properties` file. You can do this by copying the file to a backup location using the following command:

```
# cd /opt/kafka/config  
# cp server.properties server.properties_plain_1
```

- **SSL in Kafka:** By default, Kafka does not enable SSL encryption. However, we will configure SSL to secure communications between the Kafka client and the broker.

Steps to Follow:

1. **Shutdown All Kafka Brokers:** Before proceeding with the SSL configuration, ensure that all Kafka broker processes are stopped. You can stop them by:
 - Checking running processes with `jps` and killing Kafka processes using:

```
kill -9 <Kafka_process_id>
```

- Alternatively, you can use the Kafka stop script:

```
#sh /opt/kafka/bin/kafka-server-stop.sh
```

Preparation and Certificate Authority Setup

Before configuring Kafka, we must prepare our cryptographic assets. This involves creating a Certificate Authority (CA) that both the broker and clients will use to establish trust.

Install OpenSSL: The OpenSSL library is required for SSL certificate generation. Ensure it is installed on your system before proceeding.

Once you have completed these preparations, you can proceed with the SSL configuration for secure Kafka client-broker communication. Later, we will configure **Access Control Lists (ACLs)** to further secure access.

```
# dnf install openssl
```

Steps to be performed:

- Generate the Root CA (Be your own CA)
 - generate a Certificate Authority public/private key pair.
- Create a Keystore for the Broker (Generate the broker's private key)
 - Use Java's keytool to generate a private key and a temporary self-signed certificate for the broker.
- Export the Broker's Certificate(Certificate Request (CSR))- Prepare for CA signing
 - Export the broker's certificate into a file so the CA can sign it.
- Sign the Certificate by CA : Stamp the certificate
 - Use OpenSSL and your CA keys to officially sign the broker's certificate request.
- Import Certificates to Keystore(Complete the chain of trust)
 - Import the Root CA certificate into the broker's keystore, followed by the newly signed broker certificate.
- Create the Server Truststore (Tell the broker who to trust)
 - Create a truststore for the broker and import the Root CA into it. Because we are enforcing mutual TLS (mTLS), the broker needs this to verify client connections.

Here is the complete automation script.

The Automation Script

Step 1: Create the script file

Bash

```
vi /opt/scripts/generate-ssl.sh
```

Step 2: Paste the following code

Bash

```
#!/usr/bin/env bash
```

```
# Exit immediately if any command fails
set -e
```

```
# Define Variables
```

```
CER_DIR="/opt/scripts/cer"
```

```
HOSTNAME="kafka0"
```

```
KS_PASS="kafka213"
```

```
CA_PASS="EnjoyKafka"
```

```
DNAME="CN=${HOSTNAME}, OU=Unknown, O=Unknown, L=Unknown, ST=Unknown, C=Unknown"
```

```
echo "Initializing SSL directory at $CER_DIR..."
```

```
mkdir -p "$CER_DIR"
```

```
cd "$CER_DIR"
```

```
# Optional: Clean up old files if running this script multiple times
```

```
rm -f *.jks ca-cert ca-key cert-file cert-signed ca-cert.srl
```

```
echo "1. Generating the Root Certificate Authority (CA)..."
```

```
openssl req -new -x509 -keyout ca-key -out ca-cert -days 365 \
```

```
-passout pass:$CA_PASS \
```

```
-subj "/CN=${HOSTNAME}"
```

```
echo "2. Creating the Client Truststore..."
```

```
keytool -keystore client.truststore.jks -alias CARoot -import -file ca-cert \
```

```
-storepass $KS_PASS -noprompt
```

```
echo "3. Creating the Server Truststore..."
```

```
keytool -keystore server.truststore.jks -alias CARoot -import -file ca-cert \  
-storepass $KS_PASS -noprompt
```

```
echo "4. Creating the Broker Keystore & generating private key..."
```

```
keytool -keystore server.keystore.jks -alias $HOSTNAME -validity 365 -genkey -keyalg RSA \  
-storepass $KS_PASS -keypass $KS_PASS -dname "$DNAME"
```

```
echo "5. Exporting the Broker's Certificate Request (CSR)..."
```

```
keytool -keystore server.keystore.jks -alias $HOSTNAME -certreq -file cert-file \  
-storepass $KS_PASS
```

```
echo "6. Signing the Broker's Certificate with the Root CA..."
```

```
openssl x509 -req -CA ca-cert -CAkey ca-key -in cert-file -out cert-signed -days 365 -CAcreateserial \  
-passin pass:$CA_PASS
```

```
echo "7. Importing the Root CA into the Broker Keystore..."
```

```
keytool -keystore server.keystore.jks -alias CARoot -import -file ca-cert \  
-storepass $KS_PASS -noprompt
```

```
echo "8. Importing the Signed Certificate into the Broker Keystore..."
```

```
keytool -keystore server.keystore.jks -alias $HOSTNAME -import -file cert-signed \  
-storepass $KS_PASS -noprompt
```

```
echo "-----"
```

```
echo "Success! All keys, truststores, and keystores have been generated in $CER_DIR"
```

How to Run It

Save the file, make it executable, and run it:

Bash

```
chmod +x /opt/scripts/generate-ssl.sh
```

```
/opt/scripts/generate-ssl.sh
```

Result:

```
[root@kafkao /]# /opt/scripts/generate-ssl.sh
```

```
Initializing SSL directory at /opt/scripts/cer...
```

1. Generating the Root Certificate Authority (CA)...

[illegible]

2. Creating the Client Truststore...

Certificate was added to keystore

3. Creating the Server Truststore...

Certificate was added to keystore

4. Creating the Broker Keystore & generating private key...

Generating 2,048 bit RSA key pair and self-signed certificate (SHA256withRSA) with a validity of 365 days
for: CN=kafkao, OU=Unknown, O=Unknown, L=Unknown, ST=Unknown, C=Unknown

5. Exporting the Broker's Certificate Request (CSR)...

6. Signing the Broker's Certificate with the Root CA...

Certificate request self-signature ok

subject=C=Unknown, ST=Unknown, L=Unknown, O=Unknown, OU=Unknown, CN=kafka0

7. Importing the Root CA into the Broker Keystore...

Certificate was added to keystore

8. Importing the Signed Certificate into the Broker Keystore...

Certificate reply was installed in keystore

Success! All keys, truststores, and keystores have been generated in /opt/scripts/cer
[root@kafkao /]#

All the artifacts generated by the above command will be in the following folder.

`/opt/scripts/cer`

You can now immediately proceed to configuring the `server.properties` file.

`#cd /opt/scripts/cer`

Verify the server Keystore chain. Enter → kafka213

`#keytool -list -v -keystore server.keystore.jks`

Your Output should look like as shown below if everything is ok:

[root@kafkao cer]# keytool -list -v -keystore server.keystore.jks

Enter keystore password:

Keystore type: PKCS12

Keystore provider: SUN

Your keystore contains 1 entry

Alias name: kafkao

Creation date: Oct 24, 2022

Entry type: PrivateKeyEntry

Certificate chain length: 1

Certificate[1]:

Owner: CN=kafkao, OU=Unknown, O=Unknown, L=Unknown, ST=Unknown, C=Unknown

Issuer: CN=kafkao, OU=Unknown, O=Unknown, L=Unknown, ST=Unknown, C=Unknown

Serial number: 35efd4bc

Valid from: Mon Oct 24 11:00:39 IST 2022 until: Tue Oct 24 11:00:39 IST 2023

Certificate fingerprints:

SHA1: D7:0E:14:58:EB:1D:21:19:C8:CA:C4:B8:35:3A:FF:E2:7B:E2:17:A8

SHA256:

38:89:82:DF:15:7D:11:7F:80:18:AD:85:1D:C5:10:5D:34:8D:D2:74:4B:86:0F:E6:8D:88:72:2E:13:

Signature algorithm name: SHA256withRSA

Subject Public Key Algorithm: 2048-bit RSA key

Version: 3

Extensions:

#1: ObjectId: 2.5.29.14 Criticality=false

SubjectKeyIdentifier [

KeyIdentifier [

0000: 71 BF 42 54 3C 48 B6 E5 70 7A 5A 67 2C 72 5F AE q.BT<H..pzZg,r_.

0010: A2 69 62 B6 .ib.

]

]

Broker Configuration and Verification

Configure Kafka Brokers

Kafka Brokers support listening for connections on multiple ports. We need to configure the following property in server.properties, which must have one or more comma-separated values:

Server properties is in **/opt/kafka**.

Following configuration needs to be updated in the server properties file.

```
listeners=PLAINTEXT://:9092,CONTROLLER://:9093,SSL://kafka0:8092
ssl.keystore.location=/opt/scripts/cer/server.keystore.jks
ssl.keystore.password=kafka213
ssl.key.password=kafka213
ssl.truststore.location=/opt/scripts/cer/server.truststore.jks
ssl.truststore.password=kafka213
ssl.endpoint.identification.algorithm=
ssl.client.auth=required
```

Automate Broker Configuration

Create the script

Create a new script file in your scripts directory:

```
# vi /opt/scripts/configure-ssl-zk.sh
```

Paste the following code:

```
##Bash=====➔Begin[Exclude this]
```

```
#!/usr/bin/env bash
```

```
# Define the path to your server properties file
CONFIG_FILE="/opt/kafka/config/server.properties"
```

```
echo "1. Stopping the Kafka broker (if running)..."
sh /opt/kafka/bin/kafka-server-stop.sh
sleep 3 # Give it a moment to shut down
```

```
# Ensure the config file exists before modifying
if [ ! -f "$CONFIG_FILE" ]; then
    echo "Error: Configuration file not found at $CONFIG_FILE"
    exit 1
fi
```

```
echo "2. Updating the 'listeners' property..."
# Replace any existing listeners line with the new SSL configuration
sed -i 's|^listeners=.*|listeners=PLAINTEXT://:9092,SSL://kafka0:8092|' "$CONFIG_FILE"
sed -i 's|^#listeners=.*|listeners=PLAINTEXT://:9092,SSL://kafka0:8092|' "$CONFIG_FILE"
```

```

echo "3. Commenting out 'advertised.listeners'..."
# Comment out advertised.listeners if it is active
sed -i 's|^advertised.listeners=|#advertised.listeners=' "$CONFIG_FILE"

echo "4. Appending SSL configurations..."
# Remove any existing ssl.* lines to make this script safely rerun-able (idempotent)
sed -i '/^ssl\./d' "$CONFIG_FILE"

# Safely append each line one by one
echo "" >> "$CONFIG_FILE"
echo "# --- SSL Configuration ---" >> "$CONFIG_FILE"
echo "ssl.keystore.location=/opt/scripts/cer/server.keystore.jks" >> "$CONFIG_FILE"
echo "ssl.keystore.password=kafka213" >> "$CONFIG_FILE"
echo "ssl.key.password=kafka213" >> "$CONFIG_FILE"
echo "ssl.truststore.location=/opt/scripts/cer/server.truststore.jks" >> "$CONFIG_FILE"
echo "ssl.truststore.password=kafka213" >> "$CONFIG_FILE"
echo "ssl.endpoint.identification.algorithm=" >> "$CONFIG_FILE"
echo "ssl.client.auth=required" >> "$CONFIG_FILE"

echo "-----"
echo "Success! SSL properties have been added to $CONFIG_FILE."
echo "You can now start the broker using your startABroker.sh script."

```

##Bash=====→Ends[Exclude this]

You need to ensure that path to the generated keys are in **/opt/scripts/cer/** folder.
 Note: `ssl.truststore.password` is technically optional but highly recommended. If a password is not set; access to the truststore is still available, but integrity checking is disabled.

Run the Script

Apply the configurations

Save the file, make it executable, and run it:

Shutdown the broker if started earlier.

Bash

```
chmod +x /opt/scripts/configure-ssl-zk.sh  
/opt/scripts/configure-ssl-zk.sh
```

Result:

```
[root@kafka0 kraft]# /opt/scripts/configure-ssl.sh  
1. Stopping the Kafka broker (if running)...  
No kafka server to stop  
2. Updating the 'listeners' property...  
3. Commenting out 'advertised.listeners'...  
4. Appending SSL configurations...  
]
```

Start the broker now.

```
#sh /opt/scripts/startABroker.sh
```

Result:

```
[root@kafka0 scripts]# sh startABroker.sh  
Started Successfully  
[root@kafka0 scripts]# jps  
3171 Kafka  
3276 Jps
```

Once you start broker you should be able to see log details in the ***server.log***.

In the log, **/opt/kafka/logs/server.log**, there will be an entry as shown below. i.e SSL port being configured.

```
kafka.metrics.reporters = []
leader.imbalance.check.interval.seconds = 300
leader.imbalance.per.broker.percentage = 10
listener.security.protocol.map = PLAINTEXT:PLAINTEXT,SSL:SSL,SASL_PLAINTEXT:SASL_PLAINTEXT,SASL_SSL:SASL_SSL
listeners = PLAINTEXT://kafka0:9092,SSL://kafka0:8092
log.cleaner.backoff.ms = 15000
log.cleaner.deduplicate.buffer.size = 13421728
```

```
[2022-12-23 13:48:52,166] INFO Updated connection-accept-rate max connection creation rate to 2147483647 (kafka.network.ConnectionQuotas)
[2022-12-23 13:48:52,169] INFO Awaiting socket connections on kafka0:9092. (kafka.network.DataPlaneAcceptor)
[2022-12-23 13:48:52,199] INFO [SocketServer listenerType=ZK_BROKER, nodeId=0] Created data-plane acceptor and processors for endpoint : ListenerName(PLAINTEXT) (kafka.network.SocketServer)
[2022-12-23 13:48:52,200] INFO Updated connection-accept-rate max connection creation rate to 2147483647 (kafka.network.ConnectionQuotas)
[2022-12-23 13:48:52,200] INFO Awaiting socket connections on kafka0:8092. (kafka.network.DataPlaneAcceptor)
[2022-12-23 13:48:52,546] INFO [SocketServer listenerType=ZK_BROKER, nodeId=0] Created data-plane acceptor and processors for endpoint : ListenerName(SSL) (kafka.network.SocketServer)
[2022-12-23 13:48:52,554] INFO [BrokerToControllerChannelManager broker=0 name=alterPartition]: Starting (kafka.server.BrokerToControllerChannelManager)
```

Awaiting connection both in 9092(Plain) and 8092(SSL)

You can even verify using the following command to find out the listening port no.

```
#yum install lsof
#lsof -i:8092
```

```
[root@kafka0 scripts]# lsof -i:8092
COMMAND  PID  USER  FD   TYPE DEVICE SIZE/OFF NODE NAME
java     3074 root  140u  IPv4  26221      0t0  TCP kafka0:8092 (LISTEN)
[root@kafka0 scripts]#
```

To check quickly if the server keystore and truststore are setup properly you can run the following command

```
#openssl s_client -debug -connect kafka0:8092
```

(Note: TLSv1 or 1.3 should be listed under ssl.enabled.protocols or `-ts1` parameter require)
In the output of this command you should see server's certificate:

[

```
[root@kafka0 /]# openssl s_client -debug -connect kafka0:8092
Connecting to 172.18.0.2
CONNECTED(00000003)
write to 0xaaab128edca0 [0xaaab12902850] (305 bytes => 305 (0x131))
0000 - 16 03 01 01 2c 01 00 01-28 03 03 99 34 1b 16 5b  ....(4..[
0010 - 8c c3 0a 5c e0 18 58 97-04 7e 08 dd 89 bc da a1  ...X..~.....
....
0820 - 57 75 f3 36 b2 1b 82 33-1a 7d e8 43 8e 2e f8 6b  Wu.6...3}.C...k
0830 - 7f 96 ce b6 d3 f2 1b b6-d2 7e 17 4b a3 3b a3 cd  ....~.K.;..
0840 - ff 32 a8 a5 63 a1  .2..c.
Can't use SSL_get_servername
```

```

depth=1 CN=kafka0
verify error:num=19:self-signed certificate in certificate chain
verify return:1
depth=1 CN=kafka0
verify return:1
depth=0 C=Unknown, ST=Unknown, L=Unknown, O=Unknown, OU=Unknown, CN=kafka0
verify return:1
write to 0xaaab128edca0 [0xaaab12902850] (110 bytes => 110 (0x6E))
0000 - 14 03 03 00 01 01 17 03-03 00 19 c6 61 ef 64 cc .....a.d.
0010 - f3 3d 11 16 f9 14 34 bf-d3 f6 17 98 04 e4 b6 c7   ....4.....
0020 - 52 46 5b 56 17 03 03 00-45 5e c5 44 45 e2 d1 cf   RF[V....E^.DE...
0030 - 34 9e b8 41 55 83 55 39-7d 0c 98 d9 30 63 0a 1f   4..AU.U9}...0c..
0040 - 87 0d 7c 2e 71 de fa 4d-48 1f 05 52 aa 43 f8 d1   ..|.q..MH..R.C..
0050 - 13 8d 24 4b 25 35 be b3-e8 3b d4 36 86 24 43 47   ..$K%5...;.6.$CG
0060 - 1a 63 ce f7 e4 85 15 5e-39 90 bd 9f cb ef       .c.....^9.....

```

Certificate chain

```

0 s:C=Unknown, ST=Unknown, L=Unknown, O=Unknown, OU=Unknown, CN=kafka0
  i:CN=kafka0
  a:PKEY: RSA, 2048 (bit); sigalg: sha256WithRSAEncryption
  v:NotBefore: Jul 29 09:15:15 2026 GMT; NotAfter: Jul 29 09:15:15 2027 GMT
1 s:CN=kafka0
  i:CN=kafka0
  a:PKEY: RSA, 2048 (bit); sigalg: sha256WithRSAEncryption
  v:NotBefore: Jul 29 09:15:13 2026 GMT; NotAfter: Jul 29 09:15:13 2027 GMT

```

Server certificate

-----BEGIN CERTIFICATE-----

```

MIIDTDCCAjSgAwIBAgIUZuyAHm7ssAK6tOfqdxvcIKOKJAwDQYJKoZIhvcNAQEL
BQAwETEPMA0GA1UEAwGa2Fma2EwMB4XDTI2MDcyOTA5MTUxNVoXDTI3MDcyOTA5

```

```

MTUxNVowazEQMA4GA1UEBhMHVW5rbm93bjEQMA4GA1UECBMHVW5rbm93bjEQMA4G
A1UEBxMHVW5rbm93bjEQMA4GA1UEChMHVW5rbm93bjEQMA4GA1UECxMHVW5rbm93
bjEPMA0GA1UEAxMGa2Fma2EwMIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKC
AQEAzTzc†FbBu63zDYbOrUcTPPITQKV2A9obkijaZEvXK7eQoVVYm3G4eqv7UR†j
7ENiak/D5hQiRTVDrX5UAT15xoBh17cLC2pjy2E3PhT280NLAMMNrbyOSDJHJvvN
+OvPCV585FxnPhYqysKnJNn8pIqXLiHeSk8axP2V2MGi29eQLsYbPcA45kMN8Rhl
rAEN0wyfmMLX/wBibiuQqNknBdwXj†54hNqwTzjs8xauN34OnTU†nxph0fFNyAQp
pqsxAS9RU99DbbQAMP6RIKP9ExtUfVdw4Nr/W+jTYZ5xWnHJ5fOfwzxZ+3ky2RFe
z†obH12b3vSaUXj352Fdqi2XJQIDAQABo0IwQDAdBgNVHQ4EFgQUIDfDOQyPC7Hd
H7DUQLJZkfHCMjQwHwYDVR0jBBgwFoAUdu6LmOM9NVk7eJuZn8AZoDKfo/EwDQYJ
KoZIhvcNAQELBQADggEBAF8fz7CyIHsQ++zdJEPFnPBU9k2b6VxsUc4J2/g2/9Gx
icMPWpVReJN3yZLeQYOB†Fn0OSCwBZo9z0zrFMSIYlijYz0LA+fGCm4qPWIKe5M
iOY1zdAx7bklSfLZuX5hOoLaIFAmCHoD44uKipU1QQD6RYsE+2TAUVpL†DuGT1Rz
zBnd7B90E8QdU6DjIoZXs7mpXHOM87wDI†1m54X6FOUTJ98V2gqlCpje4IHsIsGb
5UQNaiuCul+552p2I8n3mRCn/xC2JJ7c2G/1Hl††xDvH1KyP2GReWVwW+b5ghF/u
xQ9dbc3EP816IEbHNmV†Tn77rEOfWu7dD55qG3SUKws=

```

-----END CERTIFICATE-----

subject=C=Unknown, ST=Unknown, L=Unknown, O=Unknown, OU=Unknown, CN=kafkaO

issuer=CN=kafkaO

Acceptable client certificate CA names

CN=kafkaO

Requested Signature Algorithms: ECDSA+SHA256:ECDSA+SHA384:ECDSA+SHA512:ed25519:ed448:RSA-
PSS+SHA256:RSA-PSS+SHA384:RSA-PSS+SHA512:rsa_pss_pss_sha256:rsa_pss_pss_sha384:rsa_pss_pss_sha512

Shared Requested Signature Algorithms: ECDSA+SHA256:ECDSA+SHA384:ECDSA+SHA512:ed25519:ed448:RSA-
PSS+SHA256:RSA-PSS+SHA384:RSA-PSS+SHA512:rsa_pss_pss_sha256:rsa_pss_pss_sha384:rsa_pss_pss_sha512

Peer signing digest: SHA256

Peer signature type: rsa_pss_rsae_sha256

Peer Temp Key: X25519, 253 bits

```

SSL handshake has read 2256 bytes and written 415 bytes
Verification error: self-signed certificate in certificate chain
---
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
Protocol: TLSv1.3
Server public key is 2048 bit
This TLS version forbids renegotiation.
Compression: NONE
Expansion: NONE
No ALPN negotiated
Early data was not sent
Verify return code: 19 (self-signed certificate in certificate chain)
---
read from 0xaaab128edca0 [0xaaab128fc2e3] (5 bytes => 5 (0x5))
0000 - 17 03 03 00 23                ....#
read from 0xaaab128edca0 [0xaaab128fc2e8] (35 bytes => 35 (0x23))
0000 - 79 ec ec 6e 2a 83 3d db-4c e9 39 2d 7a 59 be 01  y..n*.=.L.9-zY..
0010 - ec 9f 46 0c fa 0e c5 1c-2b de c7 05 db d0 0e b2  ..F.....+.....
0020 - 59 8b ed                      Y..
A00B88A6FFFF0000:error:0A00045C:SSL routines:ssl3_read_bytes:tlsv13 alert certificate
required:ssl/record/rec_layer_s3.c:918:SSL alert number 116
read from 0xaaab128edca0 [0xaaab127b0770] (16384 bytes => 0)

]

```

If the certificate does not show up or if there are any other error messages, then your keystore is not setup properly.

That is about configuring SSL on the server Side.

Secure Client Connections

Let us configure Client to connect to the broker on SSL.

Because we set **ssl.client.auth=required** on the broker, standard console producers and consumers will be rejected. We must configure the clients to present a valid certificate.

For this lab, because the client is running on the same machine as the broker, we will share the broker's keystore and truststore to authenticate the client.

Create Client Properties

Define client trust parameters

Create a file(**consumer_ssl.properties**) with the following content in **/opt/scripts/**

```
# vi /opt/scripts/consumer_ssl.properties

security.protocol=SSL
ssl.truststore.location=/opt/scripts/cer/server.truststore.jks
ssl.truststore.password=kafka213
ssl.keystore.location=/opt/scripts/cer/server.keystore.jks
ssl.keystore.password=kafka213
ssl.key.password=kafka213
```

Produce SSL Messages

Connect securely over port 8092

Check the kafka version depending on the version.

```
#/opt/kafka/bin/kafka-console-producer.sh --version
```

```
[root@kafka0 kafka]# /opt/kafka/bin/kafka-console-producer.sh --version  
3.9.0
```

Start a console producer, pointing it specifically to the SSL port (8092) and passing the security properties file.

```
#/opt/kafka/bin/kafka-console-producer.sh --bootstrap-server kafka0:8092 \  
--topic test --producer.config /opt/scripts/consumer_ssl.properties
```

or

```
#/opt/kafka/bin/kafka-console-producer.sh --bootstrap-server kafka0:8092 \  
--topic test --command-config /opt/scripts/consumer_ssl.properties
```

Send Some Messages.

Result:

[

```
[root@kafka0 kraft]# /opt/kafka/bin/kafka-console-producer.sh --bootstrap-server kafka0:8092 \  
[
```

```

--topic test --command-config /opt/scripts/consumer_ssl.properties
>Hi How are you
[2026-07-29 11:02:39,466] WARN [Producer clientId=console-producer] The metadata response from the cluster
reported a recoverable issue with correlation id 6 : {test=UNKNOWN_TOPIC_OR_PARTITION}
(org.apache.kafka.clients.NetworkClient)
>Hope you get this message
>Great
>
]
```

You should be able to consume the messages with the following commands. On a sperate terminal.

```
# /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server kafka0:8092 --topic
test --consumer.config /opt/scripts/consumer_ssl.properties --from-beginning
```

Or

```
# /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server kafka0:8092 --topic
test --command-config /opt/scripts/consumer_ssl.properties --from-beginning
```

Result:[

```
[root@kafkao /]# /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server
kafkao:8092 --topic test --consumer.config /opt/scripts/consumer_ssl.properties --from-
beginning
```

Option `--consumer.config` is deprecated and will be removed in a future version. Use `--command-config` instead.

Hi How are you

Hope you get this message

Great

]

In the Next Lab, Let us now configure ACL. Before going ahead with the next section, you need to roll back the **server.properties**.

Lab Teardown and Rollback

To prepare for the next lab (Access Control Lists), you must revert the broker back to a plain-text configuration and wipe the encrypted data from the logs.

Steps:

- Shutdown the kafka.
- Clean the kafka directories.

```
#sh /opt/kafka/bin/kafka-server-stop.sh
```

Stop all the producer and consumer console.

Steps:

- Shutdown the zookeeper and kafka.
- Clean the kafka and zookeeper directories. [If required]
- Rename the server.properties back.

----- Lab Ends Here -----

9. Securing Kafka ACL – 60 Minutes

Kafka SASL/PLAIN without SSL

We will perform the following activity in a single node broker only. We will enable ACL in a single broker.

To run a secure broker, two steps need to be performed.

1. Configure the Kafka brokers with ACL using JAAS file
2. Kafka Clients – Pass the Credentials

First, we need to let the broker know authorized users' credentials. This will be stored in a JAAS file.

1. Configure the Kafka brokers and Kafka Clients

Add a JAAS configuration file for each Kafka broker. Create a `kafka_plain_jaas.conf` file as specified below:

Prepare the `/opt/kafka/config/kafka_jaas.conf` with below contents. Use vi editor.

```
KafkaServer {  
  org.apache.kafka.common.security.plain.PlainLoginModule required  
    username="admin"  
    password="admin"
```

```
user_admin="admin"  
user_alice="alice"  
user_bob="bob"  
user_charlie="charlie";  
};
```

Let's understand the content of *kafka_plain_jaas.conf* file and how Kafka Brokers and Kafka Clients use it.

KafkaServer Section:

The **KafkaServer** section defines four users: admin, alice, bob and charlie. The properties **username** and **password** are used by the broker to initiate connections to other brokers. In this example, admin is the user for inter-broker communication. The set of properties `user_{userName}` defines the passwords for all users that connect to the broker and the broker validates all client connections including those from other brokers using these properties.

This file needs to be passed in as a JVM config option when running the broker, using -Djava.security.auth.login.config=[path_to_jaas_file].

Create an initialize scripts in `/opt/scripts/kafkasecurity.sh` file with below contents to setup `kafka_jaas.conf` file location

```
#vi /opt/scripts/kafkasecurity.sh
```

Update with the following content.

```
#!/usr/bash
export KAFKA_HOME=/opt/kafka
export ZOOKEEPER_HOME=/opt/zookeeper
export KAFKA_PLAIN_PARAMS="-
Djava.security.auth.login.config=/opt/kafka/config/kafka_jaas.conf"
export KAFKA_OPTS="$KAFKA_PLAIN_PARAMS $KAFKA_OPTS"
export PATH="$PATH:$ZOOKEEPER_HOME/bin:$KAFKA_HOME/bin:$JAVA_HOME/bin"
```

A screenshot of a terminal window with three tabs at the top: "...ata/kafka-logs (com.docker.cli) 1", "...pt/kafka/config (com.docker.cli) 2", and "...0:/opt/scripts/cer (com.docker.cli) 3". The active tab is the second one. The terminal content shows the following text:

```
#!/usr/bash
export KAFKA_HOME=/opt/kafka
export ZOOKEEPER_HOME=/opt/zookeeper
export KAFKA_PLAIN_PARAMS="-Djava.security.auth.login.config=/opt/kafka/config/kafka_jaas.conf"
export KAFKA_OPTS="$KAFKA_PLAIN_PARAMS $KAFKA_OPTS"
export PATH="$PATH:$ZOOKEEPER_HOME/bin:$KAFKA_HOME/bin:$JAVA_HOME/bin"
~
```

Use the following interpreter in case of ***bash*** interpreter issue.

```
#!/bin/bash
```

Second, the following needs to be added to the broker properties file (e.g. server.properties) it defines the accepted protocol and also the ACL authorizer used by the broker:

Step 2: Update `/opt/kafka/config/server.properties` for all brokers. In our case only one broker.

```
# Use below for SASL/PLAIN only (No SSL)
# Using SASL_PLAINTEXT as we do not have SSL
authorizer.class.name=kafka.security.authorizer.AclAuthorizer

listeners=SASL_PLAINTEXT://kafka0:9092
security.inter.broker.protocol=SASL_PLAINTEXT
sasl.mechanism.inter.broker.protocol=PLAIN
sasl.enabled.mechanisms=PLAIN
advertised.listeners=SASL_PLAINTEXT://kafka0:9092
super.users=User:admin
```

```
# listeners = PLAINTEXT://your.host.name:9092
authorizer.class.name=kafka.security.auth.SimpleAclAuthorizer
listeners=SASL_PLAINTEXT://tos.master.com:9092
security.inter.broker.protocol= SASL_PLAINTEXT
sasl.mechanism.inter.broker.protocol=PLAIN
sasl.enabled.mechanisms=PLAIN
# Hostname and port the broker will advertise to producers and consumers. If not
# set,
# it uses the value for "listeners" if configured. Otherwise, it will use the v
# value
# returned from java.net.InetAddress.getCanonicalHostName().
advertised.listeners=SASL_PLAINTEXT://tos.master.com:9092

super.users=User:admin
# Map listener names to security protocols, the default is for them to be the c
```

Comment the following line.

```
# Use SASL_PLAINTEXT as we do not have SSL
# Using SASL_PLAINTEXT as we do not have SSL
#authorizer.class.name=kafka.security.auth.SimpleAclAuthorizer
authorizer.class.name=kafka.security.authorizer.AclAuthorizer
listeners=SASL_PLAINTEXT://localhost:9092
security.inter.broker.protocol=SASL_PLAINTEXT
sasl.mechanism.inter.broker.protocol=PLAIN
sasl.enabled.mechanisms=PLAIN
advertised.listeners=SASL_PLAINTEXT://localhost:9092
super.users=User:admin

#listeners=PLAINTEXT://0.0.0.0:9092,PLAINTEXT_HOST://0.0.0.0:8082
#advertised.listeners=PLAINTEXT://localhost:9092,PLAINTEXT_HOST://localhost:8082
#listener.security.protocol.map=PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
```

Save the file and exit.

This is to initialize the environment HOME variable.

```
#cd /opt/scripts
```

```
#source ./kafkasecurity.sh
```

Restart ZK/Kafka Services or

Start our broker (from the console that executed: source ./ kafkasecurity.sh)

```
# sh startABroker.sh
```

```
[root@tos scripts]# sh startABroker.sh
ZooKeeper JMX enabled by default
Using config: /opt/zookeeper/bin/../conf/zoo.cfg
Starting zookeeper ... STARTED
Started Successfully
[root@tos scripts]# jps
2551 Kafka
2584 Jps
2286 QuorumPeerMain
[root@tos scripts]#
```

Validate the logs: ***/opt/kafka/logs/server.log*** in case there is any error.

Note : You can include sh startABroker.sh inside the kafkasecurity file in case the environments are not set properly in some OS.

```
[2023-07-01 13:48:18,026] WARN No meta.properties file under dir /opt/data/kafka-logs/sec/meta.properties (kafka.s
erver.BrokerMetadataCheckpoint)
[2023-07-01 13:48:18,061] INFO KafkaConfig values:
  advertised.listeners = SASL_PLAINTEXT://kafka0:9092
  alter.config.policy.class.name = null
  alter.log.dirs.replication.quota.window.num = 11
  alter.log.dirs.replication.quota.window.size.seconds = 1
  authorizer.class.name = kafka.security.authorizer.AclAuthorizer
  auto.create.topics.enable = true
  auto.leader.rebalance.enable = true
  background.threads = 10
  broker.heartbeat.interval.ms = 2000
  broker.id = 2
  broker.id.generation.enable = true
  broker.rack = null
  broker.session.timeout.ms = 9000
  client.quota.callback.class = null
  compression.type = producer
  connection.failed.authentication.delay.ms = 100
  connections.max.idle.ms = 600000
  connections.max.reauth.ms = 0
  control.plane.listener.name = null
```

Note:

In case of conflict in cluster ID:

Before starting the server, let us clean the zookeeper/kafka log data so that there won't be any conflict with configuration of the previous cluster (**/opt/data/zookeeper** and **/opt/data/kafka-logs**).

```
[root@tos zookeeper]# ls
version-2  zookeeper_server.pid
[root@tos zookeeper]# pwd
/opt/data/zookeeper
[root@tos zookeeper]# ls
version-2  zookeeper_server.pid
[root@tos zookeeper]# rm -fr *
[root@tos zookeeper]#
```

Delete the data folder of the zookeeper as shown above

Next Enable ACL

All the commands shown below should be executed from the path */opt/scripts*.

Create a topic:

```
#/opt/kafka/bin/kafka-topics.sh --create --bootstrap-server kafka0:9092 --  
replication-factor 1 --partitions 1 --topic plain-topic
```

```
[root@kafka0 /]# /opt/kafka/bin/kafka-topics.sh --create --bootstrap-server kafka0:9092 --replication-factor 1 --partitions 1 --topic plain-topic  
Error while executing topic command : Timed out waiting for a node assignment. Call: createTopics  
[2023-07-01 13:51:26,784] ERROR org.apache.kafka.common.errors.TimeoutException: Timed out waiting for a node assignment.  
Call: createTopics  
(kafka.admin.TopicCommand$)
```

It should generate an error like as shown above. Since, there is no permission to create a topic.

Run Kafka console producer.

Before running the Kafka console Producer configure the producer.properties file as shown:

vi producer.properties

Update with the following entries.

```
sasl.jaas.config=org.apache.kafka.common.security.plain.PlainLoginModule required \  
  username="alice" \  
  password="alice";  
security.protocol=SASL_PLAINTEXT  
sasl.mechanism=PLAIN
```

----- Code Ends Here // Exclude this line.

```
#!/opt/kafka/bin/kafka-console-producer.sh --broker-list kafka0:9092 --topic plain-  
topic --producer.config producer.properties
```

Send some messages in the producer console.

Message 1

Message 2

Message 3

```

[2020-08-30 19:14:56,662] WARN [Producer clientId=console-producer] Error while fetching metadata with correlation id 3 : {plain-topic=TOPIC_AUTHORIZATION_FAILED} (org.apache.kafka.clients.NetworkClient)
[2020-08-30 19:14:56,668] ERROR [Producer clientId=console-producer] Topic authorization failed for topics [plain-topic] (org.apache.kafka.clients.Metadata)
[2020-08-30 19:14:56,670] ERROR Error when sending message to topic plain-topic with key: null, value: 9 bytes with error: (org.apache.kafka.clients.producer.internals.ErrorLoggingCallback)
org.apache.kafka.common.errors.TopicAuthorizationException: Not authorized to access topics: [plain-topic]
>[2020-08-30 19:14:56,768] WARN [Producer clientId=console-producer] Error while fetching metadata with correlation id 4 : {plain-topic=TOPIC_AUTHORIZATION_FAILED} (org.apache.kafka.clients.NetworkClient)
[2020-08-30 19:14:56,769] ERROR [Producer clientId=console-producer] Topic authorization failed for topics [plain-topic] (org.apache.kafka.clients.Metadata)
[2020-08-30 19:14:56,769] ERROR Error when sending message to topic plain-topic with key: null, value: 9 bytes with error: (org.apache.kafka.clients.producer.internals.ErrorLoggingCallback)
org.apache.kafka.common.errors.TopicAuthorizationException: Not authorized to access topics: [plain-topic]
>[2020-08-30 19:14:56,872] WARN [Producer clientId=console-producer] Error while fetching metadata with correlation id 5 : {plain-topic=TOPIC_AUTHORIZATION_FAILED} (org.apache.kafka.clients.NetworkClient)
[2020-08-30 19:14:56,873] ERROR [Producer clientId=console-producer] Topic authorization failed for topics [plain-topic] (org.apache.kafka.clients.Metadata)
[2020-08-30 19:14:56,873] ERROR Error when sending message to topic plain-topic with key: null, value: 9 bytes with error: (org.apache.kafka.clients.producer.internals.ErrorLoggingCallback)
org.apache.kafka.common.errors.TopicAuthorizationException: Not authorized to access topics: [plain-topic]
>

```

Error since we have not authorized to access/write in the topic.

The security configuration still does not give specific permissions to our Kafka users (except for admin who is a super user). These permissions are defined using the ACL command (bin/kafka-acls.sh). To verify existing ACLs run:

```
#cd /opt/scripts  
#vi admin.properties
```

Update the following content in it.
// Begin - Exclude it

```
sasl.jaas.config=org.apache.kafka.common.security.plain.PlainLoginModule required \  
username="admin" \  
password="admin";  
security.protocol=SASL_PLAINTEXT  
sasl.mechanism=PLAIN
```

// Ends Here – Exclude it

Verify the ACL configure using the Admin credentials.

```
#!/opt/kafka/bin/kafka-acls.sh --bootstrap-server kafka0:9092 --command-config  
admin.properties --list
```

This returns no ACL definitions. You have handled authentication, but have not provided any authorization rules to define the users able run specific APIs and access certain Kafka resources.

```
[root@kafka0 scripts]#  
[root@kafka0 scripts]# /opt/kafka/bin/kafka-acls.sh --bootstrap-server kafka0:9092 --command-config admin.properties --list  
[root@kafka0 scripts]#
```

Assign the permission:

```
#/opt/kafka/bin/kafka-acls.sh --bootstrap-server kafka0:9092 --command-config  
admin.properties --add --allow-principal User:alice --operation All --topic plain-topic
```

```
[root@kafka0 config]# cd /opt/scripts  
[root@kafka0 scripts]# /opt/kafka/bin/kafka-acls.sh --bootstrap-server localhost:9092 --command-config admin.properties --  
add --allow-principal User:alice --operation All --topic plain-topic  
Adding ACLs for resource `ResourcePattern(resourceType=TOPIC, name=plain-topic, patternType=LITERAL)`:  
    (principal=User:alice, host=*, operation=ALL, permissionType=ALLOW)  
  
Current ACLs for resource `ResourcePattern(resourceType=TOPIC, name=plain-topic, patternType=LITERAL)`:  
    (principal=User:alice, host=*, operation=WRITE, permissionType=ALLOW)  
    (principal=User:alice, host=*, operation=ALL, permissionType=ALLOW)  
  
[root@kafka0 scripts]#
```

Now you can verify the acl granted to alice user

```
#/opt/kafka/bin/kafka-acls.sh --bootstrap-server kafka0:9092 --command-config  
admin.properties --list
```

```
[root@kafka0 scripts]#  
[root@kafka0 scripts]# /opt/kafka/bin/kafka-acls.sh --bootstrap-server kafka0:9092 --command-config admin.properties --list  
Current ACLs for resource `ResourcePattern(resourceType=TOPIC, name=plain-topic, patternType=LITERAL)`:  
    (principal=User:alice, host=*, operation=ALL, permissionType=ALLOW)  
  
[root@kafka0 scripts]#
```

Now you can try sending message again from the producer console. Type one or two messages in the producer console.

```
# cd /opt/scripts  
#/opt/kafka/bin/kafka-console-producer.sh --broker-list kafka0:9092 --topic plain-  
topic --producer.config producer.properties
```

Send some messages in the producer console.

Hello Message

```
[root@kafka0 scripts]# /opt/kafka/bin/kafka-console-producer.sh --broker-list localhost:9092 --topic plain-topic --producer  
.config producer.properties  
>Hello  
>Message  
>
```

Now, it should be able to send message to the topic as shown above.

Now we can try consuming these messages further from the consumer console, open a separate terminal for this.

Run Kafka console consumer

Consuming from Topic using bob user, hence grant ACL to bob.

Next, you need to let user bob consume (or fetch) from topic plain-topic using the Fetch API, as a member of the bob-group consumer group. Bob's ACL for fetching from topic test is:

Principal bob is Allowed Operation Read From Host * On Topic plain-topic .

or

```
# /opt/kafka/bin/kafka-acls.sh --bootstrap-server=kafka0:9092 -command-config admin.properties --add --allow-principal User:bob --operation Read --topic plain-topic
```

```
[root@kafka0 scripts]#  
[root@kafka0 scripts]# /opt/kafka/bin/kafka-acls.sh --bootstrap-server=kafka0:9092 -command-config admin.properties --add --allow-principal User:bob --operation Read --topic plain-topic  
Adding ACLs for resource `ResourcePattern(resourceType=TOPIC, name=plain-topic, patternType=LITERAL)`:  
    (principal=User:bob, host=*, operation=READ, permissionType=ALLOW)  
  
Current ACLs for resource `ResourcePattern(resourceType=TOPIC, name=plain-topic, patternType=LITERAL)`:  
    (principal=User:alice, host=*, operation=ALL, permissionType=ALLOW)  
    (principal=User:bob, host=*, operation=READ, permissionType=ALLOW)  
  
[root@kafka0 scripts]#
```

Bob needs a second ACL for committing offsets to group bob-group (using the OffsetCommit API):

Principal bob is Allowed Operation Read From Host * On Group bob-group.

or

```
$ /opt/kafka/bin/kafka-acls.sh --bootstrap-server=kafka0:9092 -command-config admin.properties --add --allow-principal User:bob --operation Read --group bob-group
```

```
[root@kafka0 scripts]# /opt/kafka/bin/kafka-acls.sh --bootstrap-server=kafka0:9092 -command-config admin.properties --add --allow-principal User:bob --operation Read --group bob-group
Adding ACLs for resource `ResourcePattern(resourceType=GROUP, name=bob-group, patternType=LITERAL)`:
    (principal=User:bob, host=*, operation=READ, permissionType=ALLOW)

Current ACLs for resource `ResourcePattern(resourceType=GROUP, name=bob-group, patternType=LITERAL)`:
    (principal=User:bob, host=*, operation=READ, permissionType=ALLOW)

[root@kafka0 scripts]#
```

By granting these permissions to Bob, he can now consume messages from topic test as a member of bob-group.

Before running Kafka console consumer configure the consumer.properties file as shown:

```
# vi cons.properties
```

```
sasl.jaas.config=org.apache.kafka.common.security.plain.PlainLoginModule required \
  username="bob" \
  password="bob";
security.protocol=SASL_PLAINTEXT
```

sasl.mechanism=PLAIN

```
$ /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server kafka0:9092 --topic  
plain-topic --group bob-group --from-beginning --consumer.config cons.properties
```

```
[root@kafka0 scripts]#  
[root@kafka0 scripts]# /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server kafka0:9092 --topic plain-topic --group bob-group --from-b  
eginning --consumer.config consumer.properties  
Hello Message  
Hi  
Hello  
█
```

Sent a few messages from the producer console, and you will see that messages are being consume on the console...

Describing consumer groups

Open a new console for the Charlie user.

Lastly, user charlie needs permission to retrieve committed offsets from group bob-group (using the OffsetFetch API). According to the table above, Charlie's first ACL for fetching offsets from this consumer group is:

Principal charlie is Allowed Operation Describe From Host * On Group bob-group.

OR

```
$ /opt/kafka/bin/kafka-acls.sh --bootstrap-server=kafka0:9092 -command-config
admin.properties --add --allow-principal User:charlie --operation Describe --group bob-
group
```

```
975 jps
[root@kafka0 scripts]# /opt/kafka/bin/kafka-acls.sh --bootstrap-server=kafka0:9092 -command-config admin.properties --add --allow-principal U
ser:charlie --operation Describe --group bob-group
Adding ACLs for resource `ResourcePattern(resourceType=GROUP, name=bob-group, patternType=LITERAL)`:
    (principal=User:charlie, host=*, operation=DESCRIBE, permissionType=ALLOW)

Current ACLs for resource `ResourcePattern(resourceType=GROUP, name=bob-group, patternType=LITERAL)`:
    (principal=User:bob, host=*, operation=READ, permissionType=ALLOW)
    (principal=User:charlie, host=*, operation=DESCRIBE, permissionType=ALLOW)

[root@kafka0 scripts]#
```

Principal charlie is Allowed Operation Describe From Host * On Topic test.

OR

Assign permission on the topic.

```
$ /opt/kafka/bin/kafka-acls.sh --bootstrap-server=kafka0:9092 -command-config  
admin.properties --add --allow-principal User:charlie --operation Describe --topic plain-  
topic
```

```
[root@kafka0 scripts]# /opt/kafka/bin/kafka-acls.sh --bootstrap-server=kafka0:9092 -command-config admin.properties --add --allow-principal U  
ser:charlie --operation Describe --topic plain-topic  
Adding ACLs for resource `ResourcePattern(resourceType=TOPIC, name=plain-topic, patternType=LITERAL)`:  
  (principal=User:charlie, host=*, operation=DESCRIBE, permissionType=ALLOW)  
  
Current ACLs for resource `ResourcePattern(resourceType=TOPIC, name=plain-topic, patternType=LITERAL)`:  
  (principal=User:alice, host=*, operation=ALL, permissionType=ALLOW)  
  (principal=User:charlie, host=*, operation=DESCRIBE, permissionType=ALLOW)  
  (principal=User:bob, host=*, operation=READ, permissionType=ALLOW)  
  
[root@kafka0 scripts]#
```

The following file contains the client configuration for group.

```
# vi group.properties
```

```
sasl.jaas.config=org.apache.kafka.common.security.plain.PlainLoginModule required \  
  username="charlie" \  
  password="charlie";  
security.protocol=SASL_PLAINTEXT  
sasl.mechanism=PLAIN
```

Now Charlie is able to get the proper listing of offsets in the group:

```
$ /opt/kafka/bin/kafka-consumer-groups.sh --bootstrap-server kafka0:9092 --describe
--group bob-group --command-config group.properties
```

```
[root@kafka0 scripts]# /opt/kafka/bin/kafka-consumer-groups.sh --bootstrap-server kafka0:9092 --describe --group bob-group --command-config group.properties
```

GROUP	TOPIC	PARTITION	CURRENT-OFFSET	LOG-END-OFFSET	LAG	CONSUMER-ID
bob-group	plain-topic	0	3	3	0	console-consumer-82925402-8e32-4031-9a29-b036357e4af1

```
[root@kafka0 scripts]#
```

As you can see above the details of the consumer group, Here the log end offset is showing as 3/3 that is the current offset are same with that of the end offset. This is an ideal cluster, where there is no lagging in the production and consumption.

The above ACLs grants enough permissions for this use case to run.
To summarize, configured ACLs execute the following command.

```
$ /opt/kafka/bin/kafka-acls.sh --bootstrap-server=kafka0:9092 --command-config
admin.properties --list
```

```
[root@kafka0 scripts]#
[root@kafka0 scripts]# /opt/kafka/bin/kafka-acls.sh --bootstrap-server=kafka0:9092 --command-config admin.properties --list
Current ACLs for resource `ResourcePattern(resourceType=GROUP, name=bob-group, patternType=LITERAL)`:
    (principal=User:bob, host=*, operation=READ, permissionType=ALLOW)
    (principal=User:charlie, host=*, operation=DESCRIBE, permissionType=ALLOW)

Current ACLs for resource `ResourcePattern(resourceType=TOPIC, name=plain-topic, patternType=LITERAL)`:
    (principal=User:alice, host=*, operation=ALL, permissionType=ALLOW)
    (principal=User:charlie, host=*, operation=DESCRIBE, permissionType=ALLOW)
    (principal=User:bob, host=*, operation=READ, permissionType=ALLOW)

[root@kafka0 scripts]#
```

Errors:

```
[root@tos scripts]# /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server
tos.master.com:9092 --topic plain-topic --from-beginning --consumer.config consu
mer.properties
[2018-06-27 21:37:46,223] WARN [Consumer clientId=consumer-1, groupId=console-co
nsumer-47559] Error while fetching metadata with correlation id 2 : {plain-topic
=TOPIC_AUTHORIZATION_FAILED} (org.apache.kafka.clients.NetworkClient)
[2018-06-27 21:37:46,232] ERROR Unknown error when running consumer: (kafka.too
ls.ConsoleConsumer$)
org.apache.kafka.common.errors.GroupAuthorizationException: Not authorized to ac
cess group: console-consumer-47559
```

Grant proper authorization.

Ensure that **kafkasecurity.sh** is only executed for the server start up scripts not in any of the other console.

----- Lab Ends Here-----

10. Schema Registry - Manage Schemas for Topics – 30 Minutes

Pre-requisite: Install kafka server.

Download and install confluent kafka : Schema Registry only. (Refer the Confluent Installation.)

Start the Zookeeper and Broker.

Update the following in

```
#vi /opt/confluent/etc/schema-registry/schema-registry.properties
```

```
kafkastore.bootstrap.servers=PLAINTEXT://kafka0:9092
```

Start the Registry:

```
#cd /opt/confluent/
```

```
#schema-registry-start /opt/confluent/etc/schema-registry/schema-registry.properties
```

```

WARNING: A provider io.confluent.kafka.schemaregistry.rest.resources.ModeResource registered in SERVER runtime does not implement any provider interfaces applicable in the SERVER runtime. Due to constraint configuration problems the provider io.confluent.kafka.schemaregistry.rest.resources.ModeResource will be ignored.
Sep 28, 2020 7:47:55 AM org.glassfish.jersey.internal.inject.Providers checkProviderRuntime
WARNING: A provider io.confluent.kafka.schemaregistry.rest.resources.SubjectsResource registered in SERVER runtime does not implement any provider interfaces applicable in the SERVER runtime. Due to constraint configuration problems the provider io.confluent.kafka.schemaregistry.rest.resources.SubjectsResource will be ignored.
Sep 28, 2020 7:47:55 AM org.glassfish.jersey.internal.inject.Providers checkProviderRuntime
WARNING: A provider io.confluent.kafka.schemaregistry.rest.resources.SubjectVersionsResource registered in SERVER runtime does not implement any provider interfaces applicable in the SERVER runtime. Due to constraint configuration problems the provider io.confluent.kafka.schemaregistry.rest.resources.SubjectVersionsResource will be ignored.
Sep 28, 2020 7:47:55 AM org.glassfish.jersey.internal.inject.Providers checkProviderRuntime
WARNING: A provider io.confluent.kafka.schemaregistry.rest.resources.SchemasResource registered in SERVER runtime does not implement any provider interfaces applicable in the SERVER runtime. Due to constraint configuration problems the provider io.confluent.kafka.schemaregistry.rest.resources.SchemasResource will be ignored.
[2020-09-28 07:47:56,447] INFO HV000001: Hibernate Validator 6.0.17.Final (org.hibernate.validator.internal.util.Version:21)
[2020-09-28 07:47:57,161] INFO Started o.e.j.s.ServletContextHandler@6b3871d6{/null,AVAILABLE} (org.eclipse.jetty.server.handler.ContextHandler:825)
[2020-09-28 07:47:57,215] INFO Started o.e.j.s.ServletContextHandler@aa10649{/ws,null,AVAILABLE} (org.eclipse.jetty.server.handler.ContextHandler:825)
[2020-09-28 07:47:57,272] INFO Started NetworkTrafficServerConnector@186f8716{HTTP/1.1,[http/1.1]}{0.0.0.0:8081} (org.eclipse.jetty.server.AbstractConnector:330)
[2020-09-28 07:47:57,273] INFO Started @11223ms (org.eclipse.jetty.server.Server:399)
[2020-09-28 07:47:57,275] INFO Server started, listening for requests... (io.confluent.kafka.schemaregistry.rest.SchemaRegistryMain:44)

```

Test and Verify

Create a topic and let us bind schema to this.

```

# kafka-topics --create --bootstrap-server kafka0:9092 --partitions 1 --replication-factor 1 --topic my-kafka
# kafka-topics --list --bootstrap-server kafka0:9092

```

Add a schema

Use the Schema Registry API to add a schema to the topic **my-kafka**. Subject is my-kafka-value, means the value schema of the topic – my-kafka.

```
#curl -X POST -H "Content-Type: application/vnd.schemaregistry.v1+json" --data
'{"schema":
{"\"type\": \"record\", \"name\": \"Payment\", \"namespace\": \"com.tos\", \"fields\": [{\"n
ame\": \"id\", \"type\": \"string\"}, {\"name\": \"amount\", \"type\": \"double\"}]}"
http://localhost:8081/subjects/my-kafka-value/versions
```

```
[root@kafka0 code]# kafka-topics.sh --list --bootstrap-server kafka0:9092
__consumer_offsets
__schemas
my-kafka
[root@kafka0 code]# curl -X POST -H "Content-Type: application/vnd.schemaregistry.v1+json" --data '{"schema": {"\"type\": \"record\", \"
name\": \"Payment\", \"namespace\": \"com.tos\", \"fields\": [{\"name\": \"id\", \"type\": \"string\"}, {\"name\": \"amount\", \"type\": \"double\"
}]}"}' http://localhost:8081/subjects/my-kafka-value/versions
{"id":1}[root@kafka0 code]#
```

Why this works:

- The subject name my-kafka-value follows Confluent's default TopicNameStrategy — <topic>-value for the value schema, <topic>-key for the key schema. Since the topic is my-kafka, my-kafka-value is exactly right.
- The Content-Type header is the Schema Registry-specific media type it expects.
- The inner schema field is a JSON-escaped string containing the actual Avro schema (record Payment with id and amount fields) — the double-escaping (\\") is necessary because it's a string *within* the outer JSON payload.

A successful response looks like:

```
json  
{ "id": 1 }
```

That `id` is the globally unique schema ID Schema Registry assigned.

Verify it registered:

```
# curl -X GET http://localhost:8081/schemas/ids/1
```

```
[root@kafka0 code]# curl -X GET http://localhost:8081/schemas/ids/1  
{ "schema": "{ \"type\": \"record\", \"name\": \"Payment\", \"namespace\": \"com.tos\", \"fields\": [{ \"name\": \"id\", \"type\": \"string\" }, { \"name\": \"amount\", \"type\": \"double\" }] }" } [root@kafka0 code]#
```

You can verify the topic which maintains the schema details in the broker.

```
# kafka-topics --list --bootstrap-server kafka0:9092
```

```
[root@kafka0 /]# kafka-topics.sh --list --bootstrap-server kafka0:9092  
__consumer_offsets  
_schemas  
my-kafka  
transactions  
[root@kafka0 /]#
```

List all subjects associated with a given ID

Use this two-step process to find subjects associated with a given ID:

List all the subjects.

```
#curl -X GET http://localhost:8081/subjects
```

Iterate through the output from the subject list as follows, and check for the ID in the results:

```
#curl -X GET http://localhost:8081/subjects/<INSERT SUBJECT NAME>/versions/latest
```

E.x

```
#curl -X GET http://localhost:8081/subjects/my-kafka-value/versions/latest
```

```
[root@kafka0 code]# curl -X GET http://localhost:8081/subjects
["my-kafka-value"]
[root@kafka0 code]# curl -X GET http://localhost:8081/subjects/my-kafka-value/versions/latest
{"subject":"my-kafka-value","version":2,"id":1,"schema":{"type":"record","name":"Payment","namespace":"com.tos","fields":[{"name":"id","type":"string"}, {"name":"amount","type":"double"}]}}
[root@kafka0 code]#
```

```
#curl -X GET http://localhost:8081/subjects
```

Update a schema

It will generate incompatibility schema issue, since there is no default value for the field **count** in the earlier messages.

```
#curl -X POST -H "Content-Type: application/vnd.schemaregistry.v1+json" --data
'{"schema":
{"type":"record","name":"Payment","namespace":"com.tos","fields":[{"name":"id","type":"string"}, {"name":"amount","type":"double"}, {"name":"count","type":"double"}]}' http://localhost:8081/subjects/my-kafka-value/versions
```


The key addition is "default":0 on the `count` field. Now old records missing `count` will simply be read as `count = 0` by consumers using the new schema.

Verify:

```
#curl -X GET http://localhost:8081/subjects/my-kafka-value/versions/latest
```

Delete all schema versions registered under the subject “my-kafka-value”

```
# curl -X DELETE http://localhost:8081/subjects/my-kafka-value
```

<https://docs.confluent.io/platform/current/schema-registry/develop/using.html>

----- Lab Ends Here -----

11. Kafka Connector (JDBC) - 120 Minutes

Apache Kafka Connector – Connectors are the components of Kafka that could be setup to listen the changes that happen to a data source like a file or database, and pull in those changes automatically.

Start Kafka and Zookeeper

We will use the standalone Broker this example.

```
#cd /opt/scripts
```

```
#sh startABroker.sh
```

```
[root@tos scripts]# ls
custom-reassignment.json  server2.log          stopZookeeper.sh
old.json                  start3Brokers.sh     topics-to-move.json
server0.log               startABroker.sh      zookeeper.out
server1.log               stop3Brokers.sh
[root@tos scripts]# sh startABroker.sh
ZooKeeper JMX enabled by default
Using config: /opt/zookeeper/bin/../conf/zoo.cfg
Starting zookeeper ... STARTED
Started Successfully
[root@tos scripts]# jps
2209 QuorumPeerMain
2474 Kafka
2493 Jps
[root@tos scripts]#
```

let us configure JDBC in a distributed mode.

Kafka Connect – JDBC Connector.

Download the connector from the following url. [10.9.8 Self Hosted]

Tools - <https://www.confluent.io/hub/confluentinc/kafka-connect-jdbc>

Download installation

Or download the ZIP file and extract it into one of the directories that is listed on the Connect worker's plugin.path configuration properties. This must be done on each of the installations where Connect will be run.

[Download](#)

Extract the file:

```
# yum install unzip  
# unzip confluent*.zip
```

```
[root@4f5607f478bd software]# pwd
/software
[root@4f5607f478bd software]# ls
README.md confluentinc-kafka-connect-jdbc-10.0.0 hbase-2.3.2-bin.tar log.txt master.txt
[root@4f5607f478bd software]# ls confluentinc-kafka-connect-jdbc-10.0.0/
assets doc etc lib manifest.json
[root@4f5607f478bd software]#
```

```
#mkdir -p /software/plugins/
```

Rename the folder:

```
#mv confluentinc-kafka-connect-jdbc-10.0.0/ /software/plugins/kafka-jdbc
```

Add this to the plugin path in your Connect properties file.

For example, `plugin.path=/software/plugins`. Kafka Connect finds the plugins using its plugin path.

Connect workers

Start the Connect workers with that configuration. Connect will discover all connectors defined within those plugins.

We can create `/software/connect-distributed.properties` file to specify the worker properties as follows:

Note that the `plugin.path` is the path that we need to place the library that we downloaded.

```
#vi /software/connect-distributed.properties
```

Update the following content in the above file.

A list of host/port pairs to use for establishing the initial connection to the Kafka cluster.

`bootstrap.servers=kafka0:9092`

unique name for the cluster, used in forming the Connect cluster group. Note that this must not conflict with consumer group IDs

`group.id=connect-cluster`

The converters specify the format of data in Kafka and how to translate it into Connect data.

`key.converter=org.apache.kafka.connect.json.JsonConverter`

`value.converter=org.apache.kafka.connect.json.JsonConverter`

`key.converter.schemas.enable=true`

`value.converter.schemas.enable=true`

`value.converter.schema.registry.url=http://localhost:8081`

`key.converter.schema.registry.url=http://localhost:8081`

Topic to use for storing offsets. This topic should have many partitions and be replicated and compacted.

`offset.storage.topic=connect-offsets`

`offset.storage.replication.factor=1`

Topic to use for storing connector and task configurations; note that this should be a single partition, highly replicated,

`config.storage.topic=connect-configs`

`config.storage.replication.factor=1`

Topic to use for storing statuses. This topic can have multiple partitions and should be replicated and compacted.

`status.storage.topic=connect-status`

`status.storage.replication.factor=1`

Flush much faster than normal, which is useful for testing/debugging

`offset.flush.interval.ms=10000`

`plugin.path=/software/plugins`

Note that the `plugin.path` is the path where we need to place the library that we downloaded.

Schema registry

Start the schema registry – Refer the Schema Registry lab for commands

kafka connector

Start the kafka connector

Go to the bin folder of the kafka installation or specify the full path.

```
#!/opt/kafka/bin/connect-distributed.sh /software/connect-distributed.properties
```

```

nconnectorIds=[], taskIds=[], revokedConnectorIds=[], revokedTaskIds=[], delay=0} with rebalance delay: 0 (org.apache.kafka.connect.runtime.distributed.DistributedHerder:1681)
[2020-11-15 12:59:43,196] INFO [Worker clientId=connect-1, groupId=connect-cluster] Starting connectors and tasks using config offset -1 (org.apache.kafka.connect.runtime.distributed.DistributedHerder:1208)
[2020-11-15 12:59:43,196] INFO [Worker clientId=connect-1, groupId=connect-cluster] Finished starting connectors and tasks (org.apache.kafka.connect.runtime.distributed.DistributedHerder:1236)
Nov 15, 2020 12:59:43 PM org.glassfish.jersey.internal.Errors logErrors
WARNING: The following warnings have been detected: WARNING: The (sub)resource method listLoggers in org.apache.kafka.connect.runtime.rest.resources.LoggingResource contains empty path annotation.
WARNING: The (sub)resource method createConnector in org.apache.kafka.connect.runtime.rest.resources.ConnectorsResource contains empty path annotation.
WARNING: The (sub)resource method listConnectors in org.apache.kafka.connect.runtime.rest.resources.ConnectorsResource contains empty path annotation.
WARNING: The (sub)resource method listConnectorPlugins in org.apache.kafka.connect.runtime.rest.resources.ConnectorPluginsResource contains empty path annotation.
WARNING: The (sub)resource method serverInfo in org.apache.kafka.connect.runtime.rest.resources.RootResource contains empty path annotation.

[2020-11-15 12:59:43,307] INFO Started o.e.j.s.ServletContextHandler@c6da8bb{/,null,AVAILABLE} (org.eclipse.jetty.server.handler.ContextHandler:825)
[2020-11-15 12:59:43,307] INFO REST resources initialized; server is started and ready to handle requests (org.apache.kafka.connect.runtime.rest.RestServer:319)
[2020-11-15 12:59:43,308] INFO Kafka Connect started (org.apache.kafka.connect.runtime.Connect:57)
[2020-11-15 12:59:43,419] INFO [Worker clientId=connect-1, groupId=connect-cluster] Session key updated (org.apache.kafka.connect.runtime.distributed.DistributedHerder:1570)

```

After running the connector we can confirm that connector's REST endpoint is accessible, and we can confirm that JDBC connector is in the plugin list by calling `http://localhost:8083/connector-plugins`

curl <http://localhost:8083/connector-plugins>

```

[root@4f5607f478bd software]# curl http://localhost:8083/connector-plugins
[{"class":"io.confluent.connect.jdbc.JdbcSinkConnector","type":"sink","version":"10.0.0"}, {"class":"io.confluent.connect.jdbc.JdbcSourceConnector","type":"source","version":"10.0.0"}, {"class":"org.apache.kafka.connect.file.FileStreamSinkConnector","type":"sink","version":"2.6.0"}, {"class":"org.apache.kafka.connect.file.FileStreamSourceConnector","type":"source","version":"2.6.0"}, {"class":"org.apache.kafka.connect.mirror.MirrorCheckpointConnector","type":"source","version":"1"}, {"class":"org.apache.kafka.connect.mirror.MirrorHeartbeatConnector","type":"source","version":"1"}, {"class":"org.apache.kafka.connect.mirror.MirrorSourceConnector","type":"source","version":"1"}]
[root@4f5607f478bd software]#

```

postgres db Installation.

Install postgres db. We will configure postgresdb to store data that will be transferred to kafka topic using JDBC connector.

Refer postgresdb configuration lab manual. If not given ask the instructor.

On the postgres db container. Start the postgres server if not started.

```
systemctl status postgresql
```

```
systemctl enable postgresql
```

```
systemctl start postgresql
```

```

Complete!
[root@5a1225a251e4 /]# /usr/pgsql-13/bin/postgresql-13-setup initdb
Initializing database ... OK

[root@5a1225a251e4 /]# systemctl enable postgresql-13
Created symlink from /etc/systemd/system/multi-user.target.wants/postgresql-13.service to /usr/lib/systemd/system/postgresql-13.service.
[root@5a1225a251e4 /]# systemctl start postgresql-13
[root@5a1225a251e4 /]# systemctl status postgresql-13
• postgresql-13.service - PostgreSQL 13 database server
   Loaded: loaded (/usr/lib/systemd/system/postgresql-13.service; enabled; vendor preset: disabled)
   Active: active (running) since Sun 2020-11-15 13:21:23 UTC; 6s ago
     Docs: https://www.postgresql.org/docs/13/static/
   Process: 322 ExecStartPre=/usr/pgsql-13/bin/postgresql-13-check-db-dir ${PGDATA} (code=exited, status=0/SUCCESS)
   Main PID: 327 (postmaster)
   CGroup: /docker/5a1225a251e4011bfe46ec657ce6ede8168c9c54e77cecc29df0c095aeebb37/docker/5a1225a251e4011bfe46ec657ce6ede8168c9c54e77cecc29df0c095aeebb37/system.slice/postgresql-13.service
           └─327 /usr/pgsql-13/bin/postmaster -D /var/lib/pgsql/13/data/
           └─328 postgres: logger
           └─330 postgres: checkpoint
           └─331 postgres: background writer

```

Generate some data in the postgresdb table.

```

sudo -u postgres psql
ALTER USER postgres PASSWORD 'postgres';

```

```

Complete!
[root@5a1225a251e4 /]# sudo -u postgres psql
psql (13.1)
Type "help" for help.

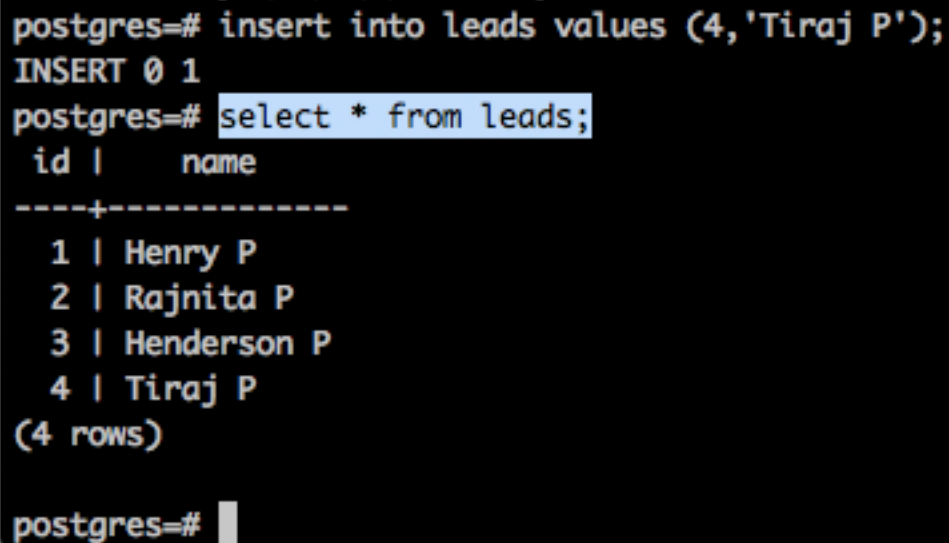
postgres=# ALTER USER postgres PASSWORD 'postgres';
ALTER ROLE
postgres=#

```

if sudo doesn't exist in your system, install (yum install sudo)

Create a table and insert few records: Execute all the below SQL commands from the Postgresdb CLI.

```
#CREATE TABLE leads (id INTEGER PRIMARY KEY, name VARCHAR);
insert into leads values (1,'Henry P');
insert into leads values (2,'Rajnita P');
insert into leads values (3,'Henderson P');
insert into leads values (4,'Tiraj P');
select * from leads;
```



```
postgres=# insert into leads values (4,'Tiraj P');
INSERT 0 1
postgres=# select * from leads;
 id |      name
----+-----
  1 | Henry P
  2 | Rajnita P
  3 | Henderson P
  4 | Tiraj P
(4 rows)

postgres=#
```

To exit : \q

On kafka.

The configuration for the plugin is stored in `/software/jdbc-source.json` file on kafka broker server. It contents is as follows: (replace localhost with container IP of postgres DB if you are using docker)

```
{
  "name": "jdbc_source_connector_postgresql_01",
  "config": {
    "connector.class": "io.confluent.connect.jdbc.JdbcSourceConnector",
    "connection.url": "jdbc:postgresql://postgresdb0:5432/postgres",
    "connection.user": "postgres",
    "connection.password": "postgres",
    "topic.prefix": "postgres-01-",
    "poll.interval.ms" : 3600000,
    "mode": "bulk"
  }
}
```

Starting the JDBC Connector – On the Kafka Node.

As we operate on distributed mode; we run the connectors by calling REST endpoints with the configuration JSON. We can specify the configuration payload from a file for curl command. The following command starts the connector. Execute the following command from the directory you have store the configuration json file.

```
#cd /software
#curl -d @"jdbc-source.json" \
-H "Content-Type: application/json" \
-X POST http://localhost:8083/connectors
```

```
[root@kafka0 software]# curl -d @"jdbc-source.json" -H "Content-Type: application/json" -X POST http://localhost:8083/connectors
{"name":"jdbc_source_connector_postgresql_02","config":{"connector.class":"io.confluent.connect.jdbc.JdbcSourceConnector","connection.url":"jdbc:po
stgresql://postgresdb:5432/postgres","connection.user":"postgres","connection.password":"postgres","topic.prefix":"postgres-01-","poll.interval.ms"
:"3600000","mode":"bulk","name":"jdbc_source_connector_postgresql_02"},"tasks":[],"type":"source"}[root@kafka0 software]#
```

On the connector console.

```
timestamp.column.name = 
timestamp.delay.interval.ms = 0
timestamp.initial = null
topic.prefix = postgres-02-
validate.non.null = true
(io.confluent.connect.jdbc.source.JdbcSourceTaskConfig:354)
[2020-11-15 16:05:29,313] INFO Using JDBC dialect PostgreSQL (io.confluent.connect.jdbc.source.JdbcSourceTask:98)
[2020-11-15 16:05:29,317] INFO Attempting to open connection #1 to PostgreSQL (io.confluent.connect.jdbc.util.CachedConnectionProvider:82)
[2020-11-15 16:05:29,383] INFO Started JDBC source task (io.confluent.connect.jdbc.source.JdbcSourceTask:257)
[2020-11-15 16:05:29,384] INFO WorkerSourceTask{id=jdbc_source_connector_postgresql_04-0} Source task finished initialization and start (org.a
pache.kafka.connect.runtime.WorkerSourceTask:233)
[2020-11-15 16:05:29,388] INFO Begin using SQL query: SELECT * FROM "public"."leads" (io.confluent.connect.jdbc.source.TableQuerier:164)
[2020-11-15 16:05:29,503] WARN [Producer clientId=connector-producer-jdbc_source_connector_postgresql_04-0] Error while fetching metadata with
correlation id 3 : {postgres-02-leads=LEADER_NOT_AVAILABLE} (org.apache.kafka.clients.NetworkClient:1073)
[2020-11-15 16:05:39,266] INFO WorkerSourceTask{id=jdbc_source_connector_postgresql_04-0} Committing offsets (org.apache.kafka.connect.runtime
.WorkerSourceTask:478)
[2020-11-15 16:05:39,268] INFO WorkerSourceTask{id=jdbc_source_connector_postgresql_04-0} flushing 0 outstanding messages for offset commit (o
rg.apache.kafka.connect.runtime.WorkerSourceTask:495)
[2020-11-15 16:05:39,295] INFO WorkerSourceTask{id=jdbc_source_connector_postgresql_04-0} Finished commitOffsets successfully in 28 ms (org.ap
ache.kafka.connect.runtime.WorkerSourceTask:574)
[2020-11-15 16:05:49,264] INFO WorkerSourceTask{id=jdbc_source_connector_postgresql_04-0} Committing offsets (org.apache.kafka.connect.runtime
.WorkerSourceTask:478)
[2020-11-15 16:05:49,265] INFO WorkerSourceTask{id=jdbc_source_connector_postgresql_04-0} flushing 0 outstanding messages for offset commit (o
rg.apache.kafka.connect.runtime.WorkerSourceTask:495)
```

Verification

We can see that postgres database table **leads** is loaded to kafka topic- **postgres-01-leads**:

```
#/opt/kafka/bin/kafka-topics.sh --list --bootstrap-server kafka0:9092
```

```
[root@kafka0 software]# /opt/kafka/bin/kafka-topics.sh --list --bootstrap-server kafka0:9092
__consumer_offsets
__schemas
connect-configs
connect-offsets
connect-status
postgres-01-leads
postgres-01-titles
[root@kafka0 software]#
```

```
#/opt/kafka/bin/kafka-topics.sh --describe --bootstrap-server kafka0:9092 --topic
postgres-01-leads
```

```
[root@kafka0 software]# /opt/kafka/bin/kafka-topics.sh --describe --bootstrap-server kafka0:9092 --topic postgres-01-leads
Topic: postgres-01-leads      TopicId: K3zU4HvbSCeM65PN0B7nUQ PartitionCount: 1      ReplicationFactor: 1      Configs: segment.bytes=1073741824
      Topic: postgres-01-leads      Partition: 0      Leader: 0      Replicas: 0      Isr: 0
[root@kafka0 software]#
```

And each row in the tables is loaded as a message. You can verify using consumer console.

```
#/opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server kafka0:9092 --topic
postgres-01-leads --from-beginning
```

```
[root@kafka0 software]# /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server kafka0:9092 --topic postgres-01-leads --from-beginning
{"schema":{"type":"struct","fields":[{"type":"int32","optional":false,"field":"id"}, {"type":"string","optional":true,"field":"name"}],"optional":false,"name":"leads"},"payload":{"id":1,"name":"Henry P"}}
{"schema":{"type":"struct","fields":[{"type":"int32","optional":false,"field":"id"}, {"type":"string","optional":true,"field":"name"}],"optional":false,"name":"leads"},"payload":{"id":2,"name":"Rajnita P"}}
{"schema":{"type":"struct","fields":[{"type":"int32","optional":false,"field":"id"}, {"type":"string","optional":true,"field":"name"}],"optional":false,"name":"leads"},"payload":{"id":3,"name":"Henderson P"}}
{"schema":{"type":"struct","fields":[{"type":"int32","optional":false,"field":"id"}, {"type":"string","optional":true,"field":"name"}],"optional":false,"name":"leads"},"payload":{"id":4,"name":"Tiraj P"}}
{"schema":{"type":"struct","fields":[{"type":"int32","optional":false,"field":"id"}, {"type":"string","optional":true,"field":"name"}],"optional":false,"name":"leads"},"payload":{"id":5,"name":"Unknown"}}
```

Single Message Transformation

Let us perform a Single Message Transformation.

Scenario: **Name** field should be changed to **FullName**

Solution: We will use the **ReplaceField** to perform the above activity.

Add another instance of connector.

Define the Single Message Transformation parameters.

The configuration for the plugin is stored in `/software/jdbc-source_smt.json` file on kafka broker server. Its contents are as follows: (replace localhost with container IP of postgres DB if you are using docker)

```
{
  "name": "jdbc_source_connector_postgresql_02",
```

```

"config": {
  "connector.class": "io.confluent.connect.jdbc.JdbcSourceConnector",
  "connection.url": "jdbc:postgresql://postgresdb:5432/postgres",
  "connection.user": "postgres",
  "connection.password": "postgres",
  "topic.prefix": "postgres-02-",
  "poll.interval.ms" : 3600000,
  "mode": "bulk",
  "transforms": "RenameField",
  "transforms.RenameField.type": "org.apache.kafka.connect.transforms.ReplaceField$Value",
  "transforms.RenameField.renames": "name:fullname"
}
}

```

In the above configuration, the field , name is replace with fullname attribute.

Start the instance

```

#curl -d @"jdbc-source_smt.json" \
-H "Content-Type: application/json" \
-X POST http://localhost:8083/connectors

```

```

[root@kafka0 software]# curl -d @"jdbc-source_smt.json" \
> -H "Content-Type: application/json" \
> -X POST http://localhost:8083/connectors
{"name":"jdbc_source_connector_postgresql_02","config":{"connector.class":"io.confluent.connect.jdbc.JdbcSourceConnector","connection.url":"jdbc:po
stgresql://postgresdb:5432/postgres","connection.user":"postgres","connection.password":"postgres","topic.prefix":"postgres-02-","poll.interval.ms"
:"3600000","mode":"bulk","transforms":"RenameField","transforms.RenameField.type":"org.apache.kafka.connect.transforms.ReplaceField$Value","transfo
rms.RenameField.renames":"name:fullname","name":"jdbc_source_connector_postgresql_02"},"tasks":[],"type":"source"}[root@kafka0 software]#

```

Verify the topic and check the message inside that topic.

```
#/opt/kafka/bin/kafka-topics.sh --list --bootstrap-server kafka0:9092
#/opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server kafka0:9092 --topic
postgres-02-leads --from-beginning
```

```
[root@kafka0 software]# /opt/kafka/bin/kafka-topics.sh --list --bootstrap-server kafka0:9092
__consumer_offsets
_schemas
connect-configs
connect-offsets
connect-status
postgres-01-leads
postgres-01-titles
postgres-02-leads
postgres-02-titles
[root@kafka0 software]# /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server kafka0:9092 --topic postgres-02-leads --from-beginning
{"schema":{"type":"struct","fields":[{"type":"int32","optional":false,"field":"id"}, {"type":"string","optional":true,"field":"fullname"}],"optional":false,"name":"leads"},"payload":{"id":1,"fullname":"Henry P"}}
{"schema":{"type":"struct","fields":[{"type":"int32","optional":false,"field":"id"}, {"type":"string","optional":true,"field":"fullname"}],"optional":false,"name":"leads"},"payload":{"id":2,"fullname":"Rajnita P"}}
{"schema":{"type":"struct","fields":[{"type":"int32","optional":false,"field":"id"}, {"type":"string","optional":true,"field":"fullname"}],"optional":false,"name":"leads"},"payload":{"id":3,"fullname":"Henderson P"}}
{"schema":{"type":"struct","fields":[{"type":"int32","optional":false,"field":"id"}, {"type":"string","optional":true,"field":"fullname"}],"optional":false,"name":"leads"},"payload":{"id":4,"fullname":"Tiraj P"}}
{"schema":{"type":"struct","fields":[{"type":"int32","optional":false,"field":"id"}, {"type":"string","optional":true,"field":"fullname"}],"optional":false,"name":"leads"},"payload":{"id":5,"fullname":"Unknown"}}
{"schema":{"type":"struct","fields":[{"type":"int32","optional":false,"field":"id"}, {"type":"string","optional":true,"field":"fullname"}],"optional":false,"name":"leads"},"payload":{"id":6,"fullname":"Another"}}
```

As shown above the message field has been transform from Name to fullname.

No of messages depend on the number of records in the table.

Information about connectors

Execute some Queries to get information about connectors:

Get a list of active connectors.

```
#curl http://localhost:8083/connectors
```

Get configuration info for a connector.

```
#curl http://localhost:8083/connectors/jdbc_source_connector_postgresql_01/config
```

```
[root@kafka0 ~]# curl http://localhost:8083/connectors
["jdbc_source_connector_postgresql_01","jdbc_source_connector_postgresql_02"]
[root@kafka0 ~]# curl http://localhost:8083/connectors/jdbc_source_connector_postgresql_01/config
{"connector.class":"io.confluent.connect.jdbc.JdbcSourceConnector","mode":"bulk","topic.prefix":"postgres-01-","connection.password":"postgres","connection.user":"postgres","poll.interval.ms":"3600000","name":"jdbc_source_connector_postgresql_01","connection.url":"jdbc:postgresql://localhost:5432/postgres"}
[root@kafka0 ~]# curl http://localhost:8083/connectors/jdbc_source_connector_postgresql_02/config
{"connector.class":"io.confluent.connect.jdbc.JdbcSourceConnector","mode":"bulk","transforms.RenameField.renames":"name:fullname","topic.prefix":"postgres-02-","connection.password":"postgres","connection.user":"postgres","poll.interval.ms":"3600000","transforms.RenameField.type":"org.apache.kafka.connect.transforms.ReplaceField$Value","transforms":"RenameField","name":"jdbc_source_connector_postgresql_02","connection.url":"jdbc:postgresql://localhost:5432/postgres"}
[root@kafka0 ~]# curl http://localhost:8083/connectors/jdbc_source_connector_postgresql_02/tasks
[{"id":{"connector":"jdbc_source_connector_postgresql_02","task":0},"config":{"connector.class":"io.confluent.connect.jdbc.JdbcSourceConnector","transforms.RenameField.renames":"name:fullname","connection.password":"postgres","transforms":"RenameField","mode":"bulk","topic.prefix":"postgres-02-","tables":"\\\"public\\\".\\\"leads\\\"","task.class":"io.confluent.connect.jdbc.source.JdbcSourceTask","connection.user":"postgres","poll.interval.ms":"3600000","transforms.RenameField.type":"org.apache.kafka.connect.transforms.ReplaceField$Value","name":"jdbc_source_connector_postgresql_02","connection.url":"jdbc:postgresql://localhost:5432/postgres"}}]
```

Retrieve details for specific tasks (JDBC Tasks) - Get a list of tasks currently running for the connector.

#curl http://localhost:8083/connectors/jdbc_source_connector_postgresql_02/tasks

```
[root@kafka0 ~]# curl http://localhost:8083/connectors/jdbc_source_connector_postgresql_02/tasks
[{"id":{"connector":"jdbc_source_connector_postgresql_02","task":0},"config":{"connector.class":"io.confluent.connect.jdbc.JdbcSourceConnector","transforms.RenameField.renames":"name:fullname","connection.password":"postgres","transforms":"RenameField","mode":"bulk","topic.prefix":"postgres-02-","tables":"\\\"public\\\".\\\"leads\\\"","task.class":"io.confluent.connect.jdbc.source.JdbcSourceTask","connection.user":"postgres","poll.interval.ms":"3600000","transforms.RenameField.type":"org.apache.kafka.connect.transforms.ReplaceField$Value","name":"jdbc_source_connector_postgresql_02","connection.url":"jdbc:postgresql://localhost:5432/postgres"}}][root@kafka0 ~]# curl http://localhost:8083/connectors/jdbc_source_connector_postgresql_01/tasks
[{"id":{"connector":"jdbc_source_connector_postgresql_01","task":0},"config":{"connector.class":"io.confluent.connect.jdbc.JdbcSourceConnector","mode":"bulk","topic.prefix":"postgres-01-","tables":"\\\"public\\\".\\\"leads\\\"","connection.password":"postgres","task.class":"io.confluent.connect.jdbc.source.JdbcSourceTask","connection.user":"postgres","poll.interval.ms":"3600000","name":"jdbc_source_connector_postgresql_01","connection.url":"jdbc:postgresql://localhost:5432/postgres"}}][root@kafka0 ~]#
```

The current status of the connector

#curl <http://localhost:8083/connectors/?expand=status>

```
[root@kafka0 ~]# curl http://localhost:8083/connectors/?expand=status
{"jdbc_source_connector_postgresql_01":{"status":{"name":"jdbc_source_connector_postgresql_01","connector":{"state":"RUNNING","worker_id":"172.18.0.2:8083"},"tasks":[{"id":0,"state":"RUNNING","worker_id":"172.18.0.2:8083"},"type":"source"]},"jdbc_source_connector_postgresql_02":{"status":{"name":"jdbc_source_connector_postgresql_02","connector":{"state":"RUNNING","worker_id":"172.18.0.2:8083"},"tasks":[{"id":0,"state":"RUNNING","worker_id":"172.18.0.2:8083"},"type":"source"]}}[root@kafka0 ~]#
```

Gets the current status of the connector, including:

- Whether it is running or restarting, or if it has failed or paused
- Which worker it is assigned to
- Error information if it has failed
- The state of all its tasks

#curl http://localhost:8083/connectors/jdbc_source_connector_postgresql_01/status

```
[root@kafka0 ~]# curl http://localhost:8083/connectors/jdbc_source_connector_postgresql_01/status
{"name":"jdbc_source_connector_postgresql_01","connector":{"state":"RUNNING","worker_id":"172.18.0.2:8083"},"tasks":[{"id":0,"state":"RUNNING","worker_id":"172.18.0.2:8083"}],"type":"source"}[root@kafka0 ~]#
```

Retrieve details for specific tasks

curl
http://localhost:8083/connectors/jdbc_source_connector_postgresql_01/tasks/0/status

```
[root@kafka0 ~]# curl http://localhost:8083/connectors/jdbc_source_connector_postgresql_01/tasks/0/status
{"id":0,"state":"RUNNING","worker_id":"172.18.0.2:8083"}[root@kafka0 ~]#
```

----- Labs End Here -----

12. Stream Kafka topic data into Elasticsearch ~ 60 Minutes

Here's a hands-on exercise for setting up an Elasticsearch Sink Connector with Kafka Connect — it builds directly on the `my-kafka` topic and schema you've already set up.

Goal: Configure a Kafka Connect Elasticsearch Sink Connector that consumes `Payment` records from `my-kafka` and indexes them into Elasticsearch in real time.

Prerequisites

- Kafka + Kafka Connect + Schema Registry running (you already have this)
- Elasticsearch running and reachable (e.g. <http://localhost:9200>)
 - Refer the Elasticsearch Lab
- The `elasticsearch-sink` connector plugin installed in your Connect worker's plugin path

Check the plugin is available:

```
curl -s http://localhost:8083/connector-plugins | jq '.[].class' | grep -i elastic
```

If it's missing, install `confluentinc-kafka-connect-elasticsearch` into your Connect `plugins` directory and restart Connect.

Download the connector ZIP **from Confluent Hub**:

<https://www.confluent.io/hub/confluentinc/kafka-connect-elasticsearch>

Version: confluentinc-kafka-connect-elasticsearch-15.1.3

```
#cp -r confluentinc-kafka-connect-elasticsearch-* /software/plugins
```

Restart/Start Kafka Connect – Distributed (Refer JDBC Connector Lab)

Now verify it.

```
#curl -s http://localhost:8083/connector-plugins | jq '[][.class] | grep -i elastic
```

Result:

```
[root@kafka0 software]# curl -s http://localhost:8083/connector-plugins | jq '[][.class] | grep -i elastic'
"io.confluent.connect.elasticsearch.ElasticsearchSinkConnector"
[root@kafka0 software]#
```

Add avro plugin also:

Check what Kafka/Connect version your worker is actually running:

```
bash
curl -s http://localhost:8083/ | jq
```

This returns something like:

```
[root@kafka0 software]# curl -s http://localhost:8083/ | jq
{
  "version": "3.9.0",
  "commit": "a60e31147e6b01ee",
  "kafka_cluster_id": "vNszaxltSTSDj3cw6bJ0yw"
}
```

Use that version to install avro converter:

➔ Kafka **3.9.0** maps to **Confluent Platform 7.9.x**.

#confluent-hub install confluentinc/kafka-connect-avro-converter:7.9.0
Select 1 option. And Yes for update.

Update the plugin path in

`/software/connect-distributed.properties`

`plugin.path=/software/plugins,/opt/confluent/share/confluent-hub-components`

Restart/Start Kafka Connect

[To Start: `/opt/kafka/bin/connect-distributed.sh /software/connect-distributed.properties`**]**

Step 1 — Confirm Elasticsearch is reachable

`#curl http://elasticsearch:9200`

You should get back a JSON blob with `cluster_name`, `version`, etc.

Step 2 — Produce some sample data onto my-kafka

Using the Avro schema you already registered:

```
kafka-avro-console-producer \  
  --bootstrap-server localhost:9092 \  
  --topic my-kafka \  
  --reader-property schema.registry.url=http://localhost:8081 \  
  --reader-property  
value.schema='{"type":"record","name":"Payment","namespace":"com.tos","fields":[{"name  
":"id","type":"string"}, {"name":"amount","type":"double"}, {"name":"count","type":"int","d  
efault":0}]}'
```

Or [Depending on the version]

```
kafka-avro-console-producer \  
  --bootstrap-server localhost:9092 \  
  --topic my-kafka \  
  --property schema.registry.url=http://localhost:8081 \  
  --property  
value.schema='{"type":"record","name":"Payment","namespace":"com.tos","fields":[{"name
```

```
":{"id","type":"string"},"name":"amount","type":"double"},"name":"count","type":"int","default":0}}}
```

Then type a few records (one JSON object per line, Ctrl+D to stop):

```
{"id":"p1","amount":100.50,"count":1}  
{"id":"p2","amount":250.00,"count":2}
```

Step 3 — Create the Elasticsearch Sink Connector

```
curl -X POST -H "Content-Type: application/json" http://localhost:8083/connectors -d '{  
  "name": "es-sink-my-kafka",  
  "config": {  
    "connector.class": "io.confluent.connect.elasticsearch.ElasticsearchSinkConnector",  
    "tasks.max": "1",  
    "topics": "my-kafka",  
    "connection.url": "http://elasticsearch:9200",  
    "type.name": "_doc",  
    "key.ignore": "true",  
    "schema.ignore": "false",  
    "key.converter": "io.confluent.connect.avro.AvroConverter",
```

```

    "key.converter.schema.registry.url": "http://localhost:8081",
    "value.converter": "io.confluent.connect.avro.AvroConverter",
    "value.converter.schema.registry.url": "http://localhost:8081",
    "behavior.on.null.values": "ignore"
  }
}
```

Result

```

{"name":"es-sink-my-kafka","config":{"connector.class":"io.confluent.connect.elasticsearch.ElasticsearchSinkConnector",
"tasks.max":"1","topics":"my-kafka","connection.url":"http://elasticsearch:9200","type.name":"_doc","key.ignore":"true","schema.ignore":"false","key.converter":"org.apache.kafka.connect.json.JsonConverter","key.converter.schema.registry.url":"http://localhost:8081","value.converter":"org.apache.kafka.connect.json.JsonConverter","value.converter.schema.registry.url":"http://localhost:8081","behavior.on.null.values":"ignore","name":"es-sink-my-kafka"},"tasks":[],"type":"sink"}[root@kafka0 software]#
```

[If required. Because of misconfiguration and to recreate it. Drop it first

```
curl -X DELETE http://localhost:8083/connectors/es-sink-my-kafka
```

]

Key config notes:

- `key.ignore: true` — Elasticsearch auto-generates document IDs instead of using the Kafka record key. Set this to `false` and use `id` from the payload as the `ES_id` if you want idempotent upserts.

- `value.converter` must match how the data was produced — since you used Avro + Schema Registry, the sink needs the Avro converter too, otherwise it can't deserialize.
- `type.name` is ignored on Elasticsearch 7+/8+ (single mapping type per index), but some connector versions still require the field.

Step 4 — Check connector status

```
curl http://localhost:8083/connectors/es-sink-my-kafka/status | jq
```

Result:

```
{
  "name": "es-sink-my-kafka",
  "connector": {
    "state": "RUNNING",
    "worker_id": "172.18.0.2:8083"
  },
  "tasks": [
    {
      "id": 0,
      "state": "RUNNING",
      "worker_id": "172.18.0.2:8083"
    }
  ],
  "type": "sink"
}
```

Look for `"state": "RUNNING"` on both the connector and its task. If the task is `FAILED`, get the trace:

```
curl http://localhost:8083/connectors/es-sink-my-kafka/status | jq '.tasks[0].trace'
```

Step 5 — Verify data landed in Elasticsearch

```
curl "http://localhost:9200/my-kafka/_search?pretty"
```

Or

```
curl "http://elasticsearch:9200/my-kafka/_search?pretty"
```

You should see your `p1` / `p2` records as documents.

Challenge tasks (extend the exercise)

1. Set `key.ignore` to `false` and `document.id.strategy` so that reprocessing the same Kafka message **updates** the existing ES doc instead of creating a duplicate.
2. Add a **Single Message Transform (SMT)** to the connector config that renames the `amount` field to `payment_amount` before it's indexed.
3. Deliberately produce a record with a schema-incompatible field and observe how the connector's `errors.tolerance` setting affects whether it dies or skips the bad record.
4. Delete the connector (`DELETE /connectors/es-sink-my-kafka`) and recreate it with `"errors.tolerance": "all"` and a dead-letter topic, then force a bad record through to see it land in the DLQ instead of crashing the task.

----- Lab Ends Here -----